

Software Requirements Specification

for

Organizational Communication System

Version 1.0

Prepared by Zosangi Renthlei

Coimbatore Institute of Technology

15th October 2026

TABLE OF CONTENTS

Table of Contents.....	ii
Revision History.....	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	2
1.4 Product Scope	3
1.5 References.....	4
2. Overall Description.....	4
2.1 Product Perspective.....	4
2.2 Product Functions.....	4
2.3 User Classes and Characteristics.....	6
2.4 Operating Environment.....	7
2.5 Design and Implementation Constraints.....	7
2.6 User Documentation.....	8
2.7 Assumptions and Dependencies.....	8
3. External Interface Requirements.....	9
3.1 User Interfaces.....	9
3.2 Hardware Interfaces.....	10
3.3 Software Interfaces.....	11
3.4 Communications Interfaces	11
4. System Features.....	12
4.1 Use Case Description.....	12
4.2 Functional Requirements.....	13
5. Other Nonfunctional Requirements.....	15
5.1 Performance Requirements.....	15
5.2 Safety Requirements.....	15
5.3 Security Requirements.....	16
5.4 Software Quality Attributes.....	16
5.5 Business Rules.....	17
6. Other Requirements	18
Appendix A: Glossary.....	19
Appendix B: Analysis Models.....	21
Appendix C: Implementation.....	24
Appendix D: To Be Determined List.....	66

Revision History

Name	Date	Reason for Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the **Organizational Communication System (OCS)**, Version 1.0. The purpose of this document is to provide a comprehensive description of the system's features, constraints, interfaces, and expected behavior.

The Organizational Communication System is designed to improve internal communication within an organization by providing employees, managers, and administrators with a centralized platform for exchanging information. The system supports communication through feedback, complaints, requests, announcements, polls, appointments, and organizational policy updates. It also includes a calendar and scheduling feature with automated reminders to improve coordination of meetings and organizational events.

This SRS serves as the primary reference for system developers, testers, project supervisors, and stakeholders throughout the software development life cycle.

1.2 Document Conventions

This document follows the IEEE Software Requirements Specification (SRS) structure to ensure consistency, clarity, and completeness.

The following conventions are used throughout this document:

- Section headings are numbered hierarchically (e.g., 1.0, 2.1, 3.2).
- Functional requirements are uniquely identified using the prefix **FR** (e.g., FR-01, FR-02).
- Non-functional requirements are uniquely identified using the prefix **NFR** (e.g., NFR-01, NFR-02).
- Use cases are identified using the prefix **UC** (e.g., UC-01, UC-02).
- The terms **shall**, **should**, and **may** are used to indicate mandatory requirements, recommended features, and optional features, respectively.
- User roles referred to in this document include **Employee**, **Manager**, and **Administrator**.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- **Project Supervisor** – to review and evaluate the proposed system requirements.
- **Software Developers** – to understand the system requirements and implement the software accordingly.
- **System Testers** – to develop test cases and verify that the implemented system satisfies the specified requirements.
- **Project Team Members** – to maintain a common understanding of the system throughout development.
- **Future Maintainers** – to understand the system architecture, functionality, and requirements for future enhancements.

The document is organized into several sections:

- **Section 1 – Introduction:** Provides an overview of the project, document purpose, scope, and references.
 - **Section 2 – Overall Description:** Describes the product perspective, user classes, operating environment, assumptions, and constraints.
 - **Section 3 – External Interface Requirements:** Defines the detailed functional and non-functional requirements of the system.
 - **Section 4 – System Features:** Includes appendices, glossary, diagrams, or supplementary material if applicable.
-

1.4 Product Scope

The Organizational Communication System (OCS) is a web-based application developed to facilitate secure, efficient, and organized communication within an organization.

The system enables employees to submit feedback, complaints, requests, and other workplace-related messages either anonymously or with their identity disclosed. Employees can also receive responses, review message history, monitor login history, and access a shared calendar that displays meetings, appointments, and automated reminders.

Managers are provided with tools to publish departmental announcements, conduct employee polls, schedule meetings and appointments, manage organizational calendars, and update departmental policies. Administrators are responsible for validating user registrations, managing user accounts and access permissions, maintaining the application, and monitoring system misuse or policy violations.

The primary objectives of the system are to:

- Improve communication between employees and management.
- Encourage honest employee feedback through optional anonymous messaging.
- Centralize organizational announcements and policy updates.
- Enhance coordination through scheduling and automated reminders.
- Improve organizational transparency and communication efficiency.
- Maintain secure access control and user management.

The Organizational Communication System supports organizational productivity by providing a centralized communication platform that reduces reliance on scattered communication channels while ensuring accountability, confidentiality, and efficient information sharing.

1.5 References

The following references were consulted during the preparation of this Software Requirements Specification:

1. **IEEE Computer Society.** *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.* Institute of Electrical and Electronics Engineers (IEEE), 1998.
2. **ISO/IEC/IEEE 29148:2018.** *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering.* International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and IEEE, 2018.
3. **Sommerville, Ian.** *Software Engineering.* 10th Edition. Pearson Education Limited, 2016.
4. **Pressman, Roger S., and Bruce R. Maxim.** *Software Engineering: A Practitioner's Approach.* 9th Edition. McGraw-Hill Education, 2019.
5. Project Documents:
 - Organizational Communication System – Use Case Diagram
 - Organizational Communication System – Software Design Documents
 - Organizational Communication System – Project Proposal (Version 1.0)

2. Overall Description

2.1 Product Perspective

The Organizational Communication System is a centralized web-based application designed to improve communication and collaboration within an organization. The system provides employees, managers, and administrators with role-specific functionalities that facilitate secure messaging, organizational announcements, scheduling, policy management, and user administration.

The system serves as a replacement for fragmented communication methods such as emails, paper notices, and third-party messaging platforms by integrating communication, scheduling, and administrative functions into a single platform.

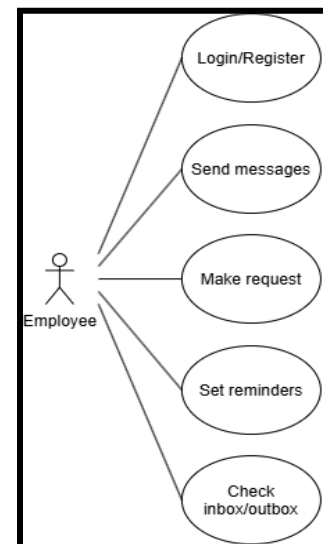
The system consists of three primary user roles:

- **Employee** – communicates with colleagues, receives announcements, manages messages, and views scheduled meetings and appointments.
- **Manager** – oversees departmental communication, schedules meetings, publishes announcements, creates polls, and manages departmental policies.
- **Administrator** – manages user accounts, validates registrations, controls user permissions, maintains application data, and handles policy violations.

2.2 Product Functions

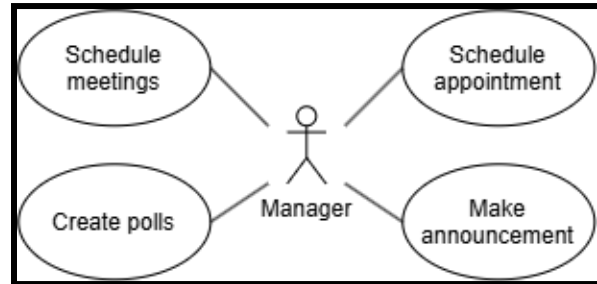
Employee Functions

- User authentication (login/logout)
- Send messages anonymously or non-anonymously
- Receive and read messages
- View sent and received message history
- View login history
- Access organizational announcements
- View department calendar
- Set reminders



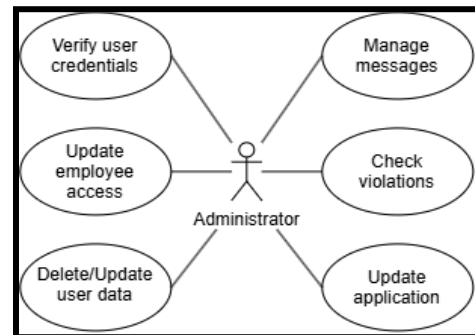
Manager Functions

- Create and publish announcements
- Create announcements with polls
- Schedule meetings and appointments
- Update the departmental calendar
- Send reminders for scheduled events
- Create, edit, and remove departmental policies



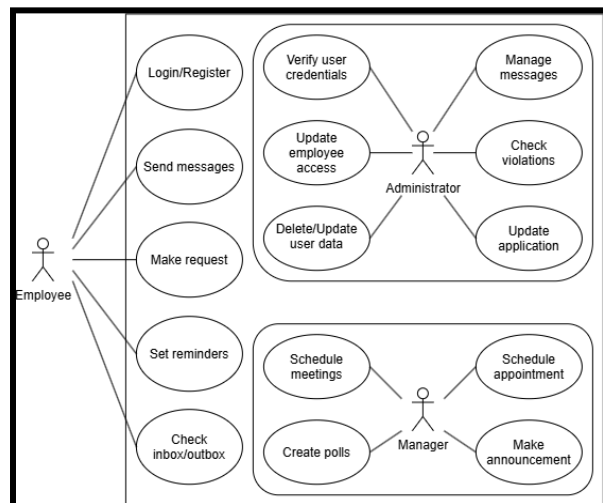
Administrator Functions

- Validate newly registered users
- Create, update, or deactivate user accounts
- Manage user roles and permissions
- Handle reports and user violations
- Maintain system data
- Perform application updates and maintenance



System Functions

- Authentication and authorization
- Notification and reminder system
- Calendar management
- Secure message storage
- Activity logging
- Audit trail generation



2.3 User Classes and Characteristics

The system supports three categories of users with different levels of access and responsibilities.

Employee

Employees are the primary users of the system. They require basic computer literacy and are responsible for communicating with colleagues, viewing announcements, accessing schedules, and managing their own messages.

Characteristics:

- Basic computer skills
- Limited system privileges
- Access restricted to personal communication and department information

Manager

Managers supervise employees within their departments and have additional privileges to distribute information and manage departmental activities.

Characteristics:

- Intermediate computer proficiency
- Responsible for department communication
- Authorized to create announcements, polls, schedules, and policies

Administrator

Administrators manage the overall system and ensure security, data integrity, and proper user access.

Characteristics:

- Advanced computer and system administration knowledge
- Full system privileges
- Responsible for maintenance, user management, and system security

2.4 Operating Environment

The Organizational Communication System will operate as a web-based application accessible through modern web browsers.

Client Environment

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Server Environment

- Web server (Nginx)
- Application server (Java)
- Relational database (MySQL)

Supported Platforms

- Windows
- macOS
- Linux

The system requires an active internet connection for communication, synchronization, and notification services.

2.5 Design and Implementation Constraints

The development of the system is subject to the following constraints:

- The system shall implement role-based access control to restrict user permissions.
- User passwords shall be securely encrypted before storage.
- Anonymous messaging shall preserve sender anonymity from recipients while maintaining administrator audit capabilities if required by organizational policy.
- The system shall maintain message and login history securely.
- The application shall support concurrent users without significant performance degradation.
- The user interface shall be responsive and accessible on desktop and mobile web browsers.
- The project shall comply with organizational privacy and security policies.

2.6 User Documentation

The system shall provide documentation to assist users in operating the application.

The documentation includes:

- User Manual
- Administrator Manual
- Installation Guide
- Quick Start Guide
- Frequently Asked Questions (FAQ)
- Help pages within the application

Documentation will describe system features, user responsibilities, navigation procedures, troubleshooting steps, and administrative operations.

2.7 Assumptions and Dependencies

The development and operation of the Organizational Communication System depend on the following assumptions:

- Users have access to a stable internet connection.
- Users possess valid organizational accounts.
- Managers are responsible for maintaining accurate schedules and announcements.
- Administrators validate user registrations before granting access.
- The organization provides the necessary server infrastructure and database services.
- Users access the application through supported web browsers.
- Notification and reminder services depend on the availability of the server and scheduling services.

3. External Interface Requirements

3.1 User Interfaces

The system will provide three distinct interfaces, one for each user role, all accessed through a common login screen.

3.1.1 Login/Authentication Interface

- Fields for username/employee ID and password, with a "Forgot Password" recovery option
- Role-based redirection to the appropriate dashboard upon successful authentication
- Clear error messaging for invalid credentials, locked accounts, or pending validation status

3.1.2 Employee Interface

- Dashboard showing recent messages, complaints, and feedback (sent and received)
- A compose screen allowing selection of recipient(s), message type (complaint, feedback, request, general message), and an anonymity toggle
- Inbox/outbox views with read/unread status indicators
- A message history log, filterable by date, type, and status
- A login history view showing timestamp, device, and location (if available) of past sessions
- A read-only calendar view showing company-wide and department-specific events
- Notification indicators (badges/alerts) for new messages or calendar updates

3.1.3 Manager Interface

- All Employee-level interface elements, scoped to their department
- An announcement composer with options to target specific departments or the whole company
- A poll creation tool with configurable question types (single-choice, multiple-choice), deadlines, and anonymity settings
- A meeting/appointment scheduler with date/time picker, invitee selection, and conflict detection against the department calendar
- Calendar management controls (add/edit/delete department events)
- A policy management screen for uploading, editing, and publishing department policy documents

3.1.4 Administrator Interface

- A user validation queue showing pending registrations with approve/reject actions
- A user management screen for editing roles, permissions, and account status (active/suspended/deactivated)
- A violation handling dashboard listing reported issues, with tools to investigate, escalate, or close cases
- An application configuration/update panel for managing system-wide settings and announcements
- Audit log access across all users' login and message history for compliance and moderation purposes

3.1.5 General UI Requirements

- Responsive design supporting desktop and mobile screen sizes
 - Consistent navigation structure and visual language across all three role-based interfaces
 - Accessibility compliance (readable contrast ratios, keyboard navigation, screen-reader-friendly labels)
 - Multi-language support may be considered as a future enhancement (optional)
-

3.2 Hardware Interfaces

The application is a web/mobile-based system and does not require specialized hardware. Standard interfaces are as follows:

- **Client-side devices:** Desktop computers, laptops, tablets, or smartphones with standard input devices (keyboard, mouse, touchscreen) and a functioning display
- **Network hardware:** Standard Wi-Fi or cellular network adapters for internet connectivity
- **Server-side hardware:** Application and database servers with sufficient CPU, RAM, and storage to handle concurrent user sessions, message storage, and calendar data (specific sizing to be determined during deployment planning)
- No biometric scanners, external peripherals, or proprietary hardware are required for standard operation

3.3 Software Interfaces

Component	Requirement
Client Operating System	Windows, macOS, Linux, Android, or iOS (any OS supporting a modern web browser or the mobile app)
Web Browser	Latest two major versions of Chrome, Firefox, Safari, or Edge
Backend Runtime	Node.js / Java / Python (to be finalized per implementation)
Database Management System	PostgreSQL or MySQL for relational data (users, messages, calendar events, policies)
Authentication Service	Internal auth module or third-party identity provider (e.g., OAuth2) for login/session management
Notification Service	Email delivery service (e.g., SMTP provider) and/or push notification service for real-time alerts
API Layer	RESTful API exposing endpoints consumed by the Employee, Manager, and Admin client applications
Version Control/Deployment	Git for source control; CI/CD pipeline for application updates

The system's internal services (User Management, Communication, Scheduling, Admin Operations) interface with each other through internal API calls, and with the database through standard database drivers/ORM libraries.

3.4 Communications Interfaces

- **Protocol:** All client-server communication occurs over HTTPS, with TLS encryption enforced for every request, including anonymous complaint submissions, to protect sender identity in transit
- **API Communication:** RESTful API calls using JSON as the data interchange format between client applications and backend services
- **Real-Time Updates:** WebSocket or a polling mechanism for delivering real-time notifications (new messages, poll results, announcement alerts) without requiring manual refresh
- **Email/SMS Notifications:** Outbound communication via SMTP (or an SMS gateway, if implemented) to notify users of account validation status, new messages, or scheduled meetings, without exposing anonymous senders' identities
- **Session Management:** Token-based session handling (e.g., JWT) transmitted over secure headers for authenticated requests

4. System Features

4.1 Use Case Description

Primary Actor: Employee

Precondition:

- Employees have access to the system.
- Registration is approved by the administrator.

Postcondition:

- Messages are stored in the system.
- Profile changes are updated.
- Login activity is recorded.
- Communication history is maintained.

Trigger: Employee wants to access the communication system.

Main Success Scenario:

1. Employee registers (if new).
2. Administrator approves the registration.
3. Employee logs into the system.
4. Employee updates profile if required.
5. Employee creates a message and selects its type (feedback, complaint, request).
6. System delivers the message to the appropriate recipient.
7. Employee receives responses or notifications.
8. Employee views message history and login history.
9. Employee logs out.

Exception Scenario:

- Registration is rejected by the administrator.
 - Invalid login credentials are entered.
 - Required message fields are missing.
 - System temporarily unavailable.
-

Primary Actor: Manager

Precondition:

- Manager account exists and is active.
- Manager is authenticated.

Postcondition:

- Announcements are published.
- Appointments are scheduled.

Trigger: Manager logs in to perform management activities.

Main Success Scenario:

1. Manager logs in.
2. Manager creates and publishes announcements.
3. Manager schedules appointments with employees.
4. Employees receive notifications regarding these.
5. Manager logs out.

Exception Scenario:

- Appointments conflict with existing schedule.
 - Session expires.
-

Primary Actor: Administrator

Precondition:

- Administrator account exists.
- Administrator is logged in.

Postcondition:

- User accounts are approved or rejected.
- User records are updated.
- System information is maintained.

Trigger: Administrator accesses the administration panel.

Main Success Scenario:

1. Administrator logs into the system.
2. Reviews pending registration requests.
3. Approves or rejects requests.
4. Updates employee information.
5. Performs application maintenance/updates.
6. Reviews violations and takes necessary action.
7. Logs out.

Exception Scenario:

- Employee record not found.
 - Application update fails.
-

4.2 Functional Requirements

This section translates the use case descriptions in Section 4.1 into specific, verifiable functional requirements, grouped by actor.

Employee

- **FR-1:** The system shall allow a new employee to register an account.
- **FR-2:** The system shall place a new registration in a pending state until the administrator approves or rejects it.

- **FR-3:** The system shall allow a registered employee to log in using valid credentials.
- **FR-4:** The system shall reject login attempts with invalid credentials.
- **FR-5:** The system shall allow an employee to update their profile information.
- **FR-6:** The system shall allow an employee to compose a message and classify it by type (feedback, complaint, or request).
- **FR-7:** The system shall reject message submission if required fields are missing.
- **FR-8:** The system shall deliver a submitted message to the intended recipient.
- **FR-9:** The system shall notify an employee of responses received to their messages.
- **FR-10:** The system shall allow an employee to view their message history.
- **FR-11:** The system shall allow an employee to view their login history.
- **FR-12:** The system shall allow an employee to log out, ending their session.
- **FR-13:** The system shall display an appropriate error if the system is temporarily unavailable during any of the above actions.

Manager

- **FR-14:** The system shall allow a manager to log in using valid, active credentials.
- **FR-15:** The system shall allow a manager to create and publish announcements.
- **FR-16:** The system shall allow a manager to schedule appointments with employees.
- **FR-17:** The system shall detect and prevent scheduling an appointment that conflicts with an existing one.
- **FR-18:** The system shall notify employees when an announcement is published or an appointment is scheduled involving them.
- **FR-19:** The system shall terminate a manager's session if it expires, requiring re-authentication.
- **FR-20:** The system shall allow a manager to log out, ending their session.

Administrator

- **FR-21:** The system shall allow an administrator to log into the administration panel.
- **FR-22:** The system shall allow an administrator to review pending user registration requests.
- **FR-23:** The system shall allow an administrator to approve or reject a pending registration.
- **FR-24:** The system shall allow an administrator to update employee records.
- **FR-25:** The system shall display an appropriate error if a referenced employee record is not found.
- **FR-26:** The system shall allow an administrator to perform application maintenance and updates.
- **FR-27:** The system shall notify the administrator if an application update fails.
- **FR-28:** The system shall allow an administrator to review reported violations and take corresponding action.
- **FR-29:** The system shall allow an administrator to log out, ending their session.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **PR-1:** The system shall load any page (dashboard, inbox, calendar, etc.) within 3 seconds under normal network conditions.
 - **PR-2:** The system shall support at least 100 concurrent active user sessions without noticeable degradation in response time.
 - **PR-3:** A message, once sent, shall be delivered to the recipient's inbox within 5 seconds.
 - **PR-4:** The system shall process login/authentication requests within 2 seconds.
 - **PR-5:** The system shall be able to store and retrieve at least 10,000 messages and 1,000 user accounts without a measurable drop in performance.
 - **PR-6:** Calendar and poll data shall refresh to reflect new entries within 5 seconds of submission, without requiring a full page reload where real-time updates are implemented.
-

5.2 Safety Requirements

- **SR-1:** The system shall not permanently delete user data (accounts, messages, calendar events) as a result of a single accidental action; destructive actions shall require confirmation.
- **SR-2:** The system shall preserve message and login history even if a user's account is suspended or deactivated, for accountability and audit purposes.
- **SR-3:** In the event of a system crash or unexpected shutdown, no partially submitted data (e.g., a half-completed message or registration) shall be saved in a corrupted or inconsistent state.
- **SR-4:** The system shall maintain a backup of critical data (user accounts, messages, calendar, policies) on a regular schedule to prevent permanent data loss.
- **SR-5:** The system shall log all administrator actions that affect user accounts (approval, rejection, suspension, role changes) to maintain an audit trail in case of disputes or misuse.

5.3 Security Requirements

- **SC-1:** All client-server communication shall be encrypted using HTTPS/TLS, including anonymous complaint submissions, to protect data in transit.
 - **SC-2:** Passwords shall be stored in hashed (not plain-text) form using an industry-standard hashing algorithm.
 - **SC-3:** The system shall enforce role-based access control (RBAC), restricting each user to only the features and data permitted for their role (Employee, Manager, Administrator).
 - **SC-4:** The identity of a sender who submits a message anonymously shall not be exposed to the recipient through the user interface, database exposure, or notification content.
 - **SC-5:** The system shall automatically terminate (expire) an inactive user session after a defined period of inactivity, requiring re-authentication.
 - **SC-6:** The system shall lock or flag an account after a defined number of consecutive failed login attempts, to mitigate brute-force attacks.
 - **SC-7:** Only an authenticated Administrator shall be able to access user validation, role management, and violation-handling functions.
 - **SC-8:** The system shall validate and sanitize all user input to prevent injection attacks (e.g., SQL injection, script injection).
-

5.4 Software Quality Attributes

- **Usability:** The system shall provide a consistent, intuitive interface across all three role-based views, requiring no more than a brief orientation for a new user to perform core tasks (send a message, view the calendar, log in).
- **Reliability:** The system shall maintain at least 99% uptime during business hours, excluding scheduled maintenance windows.
- **Maintainability:** The system shall be built with modular components (e.g., separate services for messaging, scheduling, and user management) to allow features to be updated or extended independently.
- **Scalability:** The system architecture shall support future growth in user count and feature set (e.g., additional departments, message types) without requiring a full redesign.
- **Availability:** The system shall be accessible from standard web browsers and, where applicable, mobile devices, without requiring specialized client software.
- **Portability:** The system shall function consistently across the latest two major versions of Chrome, Firefox, Safari, and Edge, as specified in Section 3.3.

5.5 Business Rules

- **BR-1:** A new user account shall remain in "pending" status and shall not be granted system access until explicitly approved by an Administrator.
- **BR-2:** Only Managers and Administrators may create or modify department/company calendar events, announcements, polls, and meetings.
- **BR-3:** Only Administrators may approve or reject registrations, change user roles, or suspend/reactivate accounts.
- **BR-4:** An employee may choose to submit a message anonymously; once submitted, the anonymity setting cannot be changed or revoked.
- **BR-5:** A user may cast only one vote per poll.
- **BR-6:** A suspended or deactivated account shall be denied login access until reactivated by an Administrator, regardless of correct credentials.
- **BR-7:** Reported violations shall remain in "open" status until explicitly marked "resolved" by an Administrator.

6. Other Requirements

6.1 Legal and Regulatory Requirements

- **LR-1:** The system shall comply with applicable data privacy regulations (e.g., the Data Privacy Act, or equivalent regional data protection law) regarding the collection, storage, and processing of employee personal information.
 - **LR-2:** The system's anonymous complaint feature shall be designed to genuinely protect sender identity, consistent with whistleblower protection principles, so that an anonymous complainant cannot be identified through system logs, database records, or administrator tools without a formal legal process.
 - **LR-3:** The organization deploying the system shall retain the right to disclose an anonymous sender's identity only where legally compelled (e.g., court order), and the system shall support this as an exceptional, audited action rather than routine access.
 - **LR-4:** The system shall comply with the organization's internal data governance and records-retention policies.
-

6.2 Data Retention and Archiving

- **DR-1:** Message, login, and violation records shall be retained for a minimum period defined by the organization's policy (e.g., one year) before archiving or deletion.
 - **DR-2:** Archived data shall remain retrievable by an Administrator for audit or compliance purposes, even after being moved out of active storage.
 - **DR-3:** Upon a user's permanent removal from the organization, their account shall be deactivated rather than deleted, to preserve messages/audit trails tied to their activity.
-

6.3 Installation and Deployment Requirements

- **ID-1:** The system shall be deployable on standard institutional or cloud server infrastructure without requiring proprietary or specialized hosting environments.
- **ID-2:** Installation and initial setup (including creation of the first Administrator account) shall be completable by a technically competent staff member without vendor intervention.
- **ID-3:** The system shall support incremental updates/patches to be applied by the Administrator without requiring full reinstallation or data loss, consistent with the "update the application" responsibility defined for the Administrator role.

Appendix A: Glossary

Administrator — The user role responsible for validating new user registrations, managing user roles and access privileges, handling reported violations, and maintaining/updating the application.

Anonymity — A property of a submitted message in which the sender's identity is withheld from the recipient, while still being deliverable and traceable by the system for administrative purposes.

Announcement — A broadcast message created by a Manager, visible to employees within a department or company-wide.

Audit Trail — A chronological record of system actions (e.g., login activity, administrator actions on user accounts) maintained for accountability and compliance purposes.

Authentication — The process of verifying a user's identity, typically through a username and password, before granting access to the system.

Complaint — A type of message submitted by an employee expressing a grievance or concern, which may be sent anonymously.

Department — An organizational subdivision to which employees and managers may belong, used to scope calendar events and policies where department-level separation is implemented.

Employee — The user role that can send and receive messages (including anonymous ones), view the calendar, and access personal message and login history.

Feedback — A type of message submitted by an employee to share input, suggestions, or opinions with a peer or superior.

Functional Requirement (FR) — A statement describing a specific behavior or function the system must perform, derived from a use case.

IEEE 830 — A standard format/template for Software Requirements Specification (SRS) documents, which this document follows.

Meeting/Appointment — A scheduled event created by a Manager, involving one or more invited employees, with a defined date, time, and title.

Message — A unit of communication sent by a user to one or more recipients, which may be categorized as feedback, complaint, request, or general communication.

Nonfunctional Requirement (NFR) — A requirement describing a quality attribute or constraint of the system (e.g., performance, security, usability) rather than a specific function.

Policy — A department- or company-level rule or guideline document, manageable by Managers.

Pending Status — The state assigned to a newly registered user account before an Administrator approves or rejects it; a pending account cannot log in.

Request — A type of message submitted by an employee asking a peer or superior to take a specific action.

Role-Based Access Control (RBAC) — A security model that restricts system features and data access based on a user's assigned role (Employee, Manager, or Administrator).

Session — A period of authenticated interaction between a user and the system, beginning at login and ending at logout or timeout.

Session Token — A unique identifier issued to a user upon successful login, used to maintain their authenticated session without requiring repeated credential entry.

SRS (Software Requirements Specification) — This document; a formal specification describing the functional and nonfunctional requirements of the system to be built.

Suspended Account — A user account that has been temporarily deactivated by an Administrator, preventing login until reactivated.

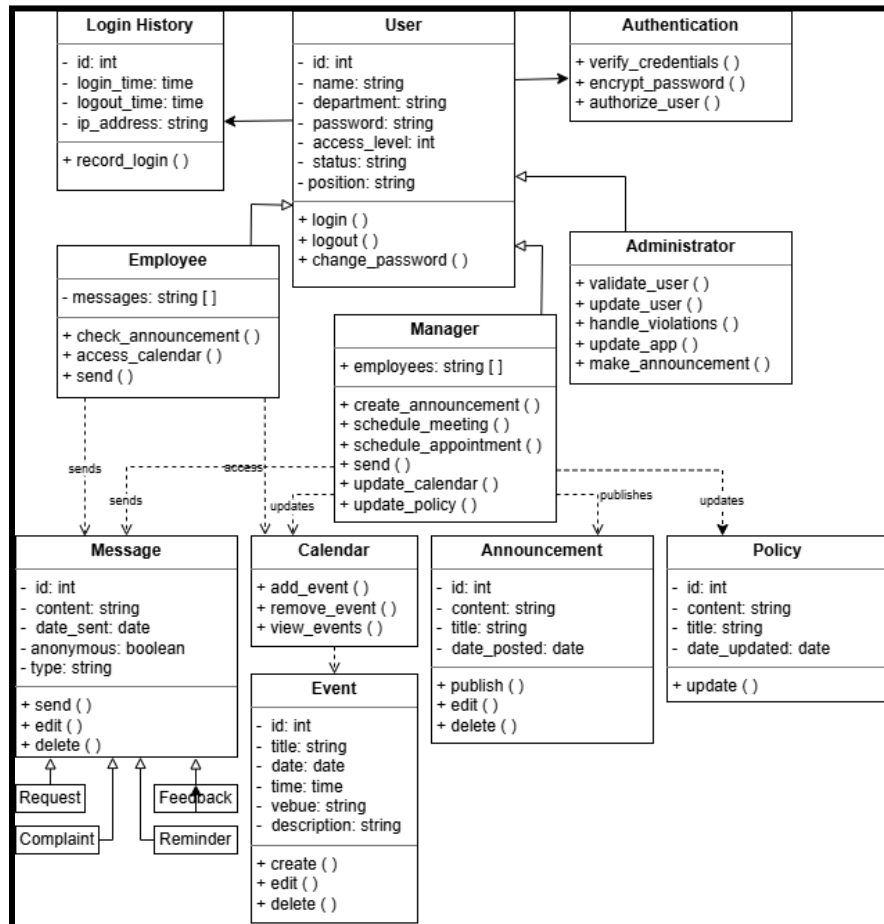
Use Case — A structured description of a way a specific actor (Employee, Manager, or Administrator) interacts with the system to achieve a goal, including its trigger, main success scenario, and exception scenarios.

UML (Unified Modeling Language) — A standardized visual modeling language used in this document to represent system behavior and structure (e.g., activity, sequence, state chart, component, and deployment diagrams).

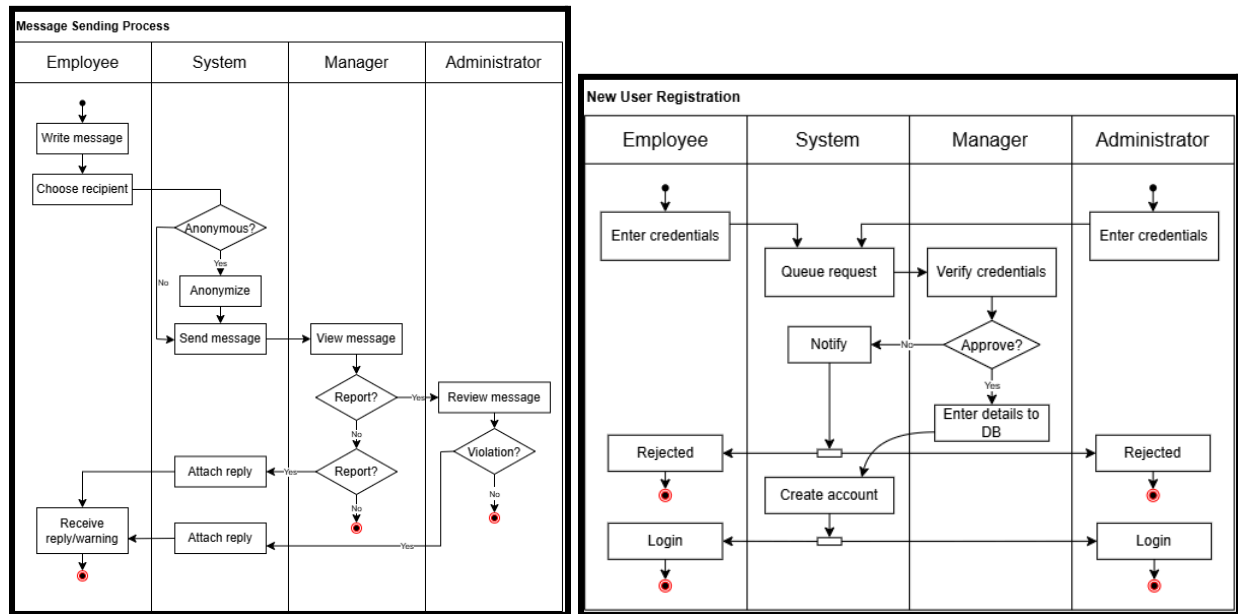
Violation — A reported issue or breach of policy submitted by an Employee or Manager, reviewed and resolved by an Administrator.

Appendix B: Analysis Models

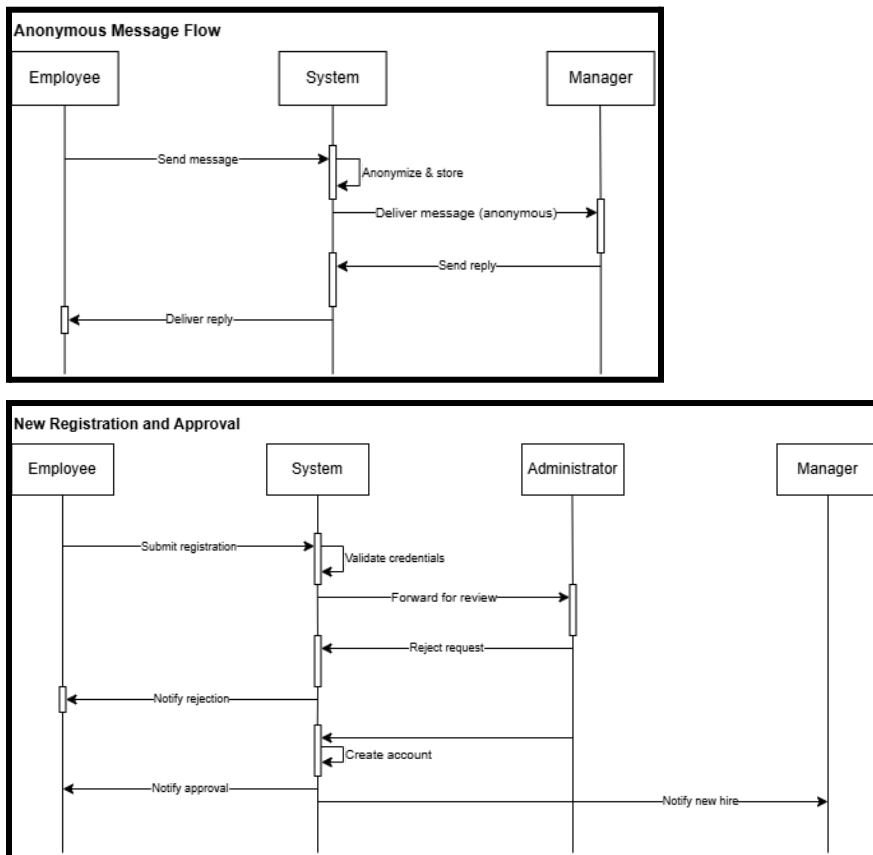
UML Class Diagram:



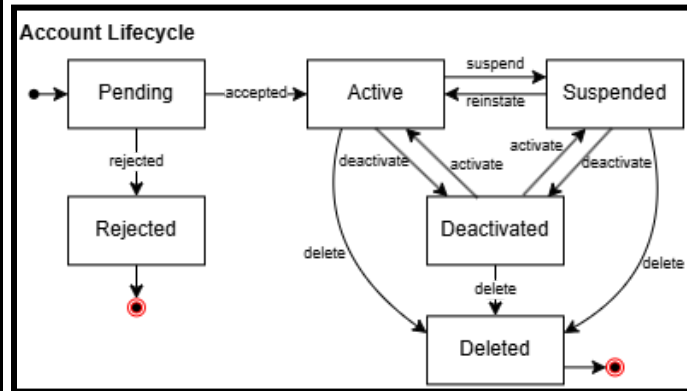
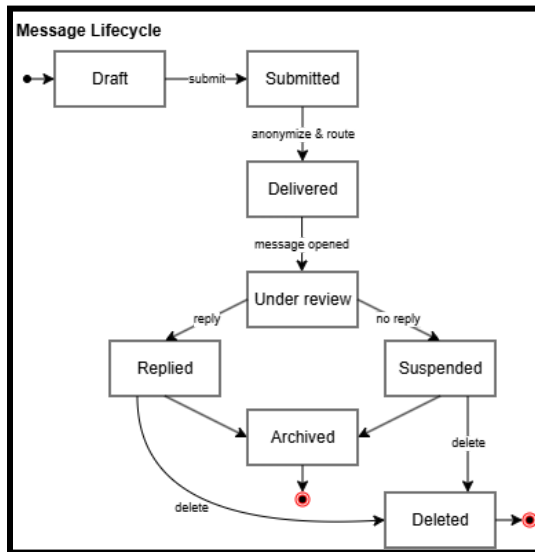
UML Activity Diagram:



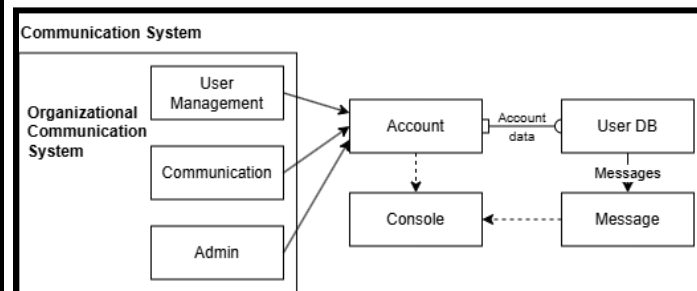
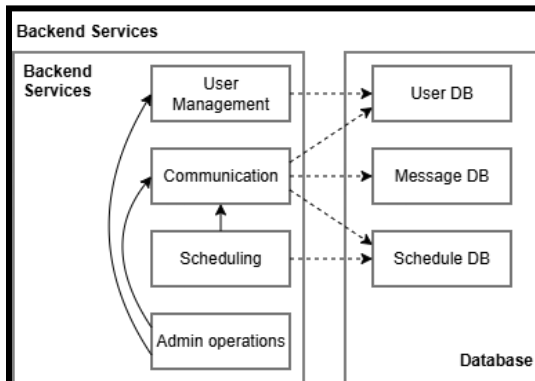
UML Sequence Diagram:



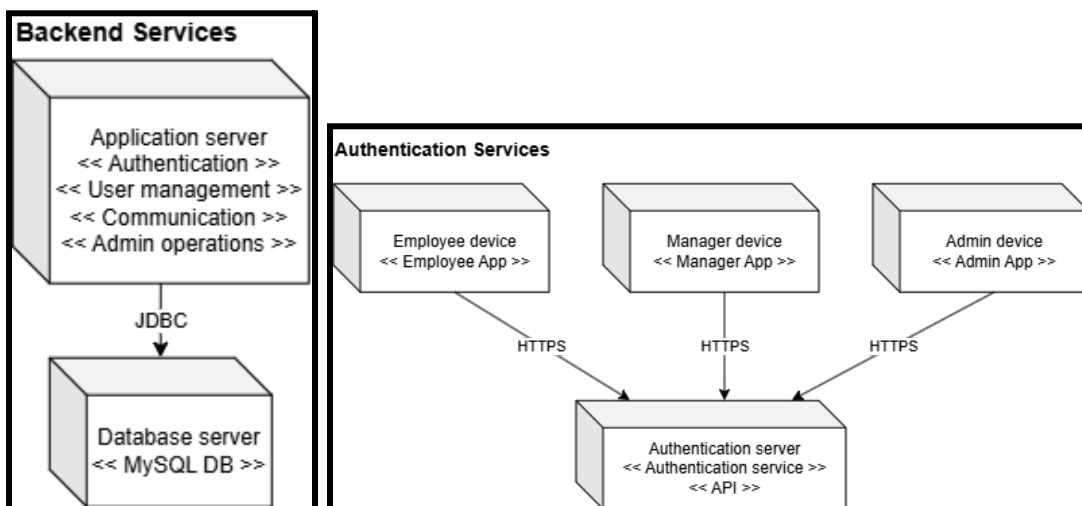
UML State Chart Diagram:



UML Component Diagram:



UML Deployment Diagram:



Appendix C: Implementation

schema.sql:

```
CREATE DATABASE IF NOT EXISTS ocs
CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE ocs;
```

```
CREATE TABLE IF NOT EXISTS users (
  id          INT AUTO_INCREMENT
PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  username    VARCHAR(50) NOT NULL
UNIQUE,
  password_hash VARCHAR(255) NOT
NULL,
  role
ENUM('employee','manager','admin') NOT
NULL DEFAULT 'employee',
  status
ENUM('pending','active','suspended','rejecte
d','deactivated','locked') NOT NULL
DEFAULT 'pending',
  department  VARCHAR(100) NOT
NULL DEFAULT "",
  failed_attempts INT NOT NULL DEFAULT
0,
  created_at   TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS messages
(
  id          INT AUTO_INCREMENT
PRIMARY KEY,
  sender_id   INT NOT NULL,
  recipient_id INT NOT NULL,
  type
ENUM('feedback','complaint','request','gene
ral') NOT NULL DEFAULT 'general',
  body        TEXT NOT NULL,
```

```
  anonymous   BOOLEAN NOT NULL
DEFAULT FALSE,
  sent_at     TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sender_id)
REFERENCES users(id),
  FOREIGN KEY (recipient_id)
REFERENCES users(id),
  INDEX idx_msg_recipient (recipient_id),
  INDEX idx_msg_sender (sender_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS
calendar_events (
  id          INT AUTO_INCREMENT
PRIMARY KEY,
  title       VARCHAR(200) NOT NULL,
  event_date  DATE NOT NULL,
  description TEXT,
  created_by  INT NULL,
  FOREIGN KEY (created_by)
REFERENCES users(id)
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS
announcements (
  id          INT AUTO_INCREMENT PRIMARY
KEY,
  title       VARCHAR(200) NOT NULL,
  body        TEXT NOT NULL,
  created_by  INT NULL,
  posted_at   TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (created_by)
REFERENCES users(id)
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS polls (
```

```

    id      INT AUTO_INCREMENT
PRIMARY KEY,
    question VARCHAR(300) NOT NULL,
    created_by INT NULL,
    created_at TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by)
REFERENCES users(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS
poll_options (
    id      INT AUTO_INCREMENT PRIMARY
KEY,
    poll_id INT NOT NULL,
    label   VARCHAR(200) NOT NULL,
    FOREIGN KEY (poll_id) REFERENCES
polls(id) ON DELETE CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS poll_votes
(
    poll_id INT NOT NULL,
    user_id INT NOT NULL,
    option_id INT NOT NULL,
    voted_at TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (poll_id, user_id),
    FOREIGN KEY (poll_id) REFERENCES
polls(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES
users(id),
    FOREIGN KEY (option_id) REFERENCES
poll_options(id) ON DELETE CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS meetings
(
    id      INT AUTO_INCREMENT
PRIMARY KEY,
    title   VARCHAR(200) NOT NULL,
    meeting_time DATETIME NOT NULL,
    invitees VARCHAR(500) NOT NULL
DEFAULT "",

```

```

    created_by INT NOT NULL,
    FOREIGN KEY (created_by)
REFERENCES users(id),
    INDEX idx_meeting_time (meeting_time)
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS violations
(
    id      INT AUTO_INCREMENT
PRIMARY KEY,
    reporter_id INT NOT NULL,
    description TEXT NOT NULL,
    status     ENUM('open','resolved') NOT
NULL DEFAULT 'open',
    reported_at TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (reporter_id)
REFERENCES users(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS
login_history (
    id      INT AUTO_INCREMENT
PRIMARY KEY,
    user_id INT NOT NULL,
    login_time TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,
    ip_address VARCHAR(64) NOT NULL
DEFAULT "",
    FOREIGN KEY (user_id) REFERENCES
users(id),
    INDEX idx_login_user (user_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS audit_log
(
    id      INT AUTO_INCREMENT
PRIMARY KEY,
    admin_id INT NOT NULL,
    action   VARCHAR(60) NOT NULL,
    details  VARCHAR(300) NOT NULL
DEFAULT "",
    created_at TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP,

```

```
FOREIGN KEY (admin_id) REFERENCES
users(id)
) ENGINE=InnoDB;
```

Main.java:

```
package com.ocs;

import com.ocs.dao.UserDao;
import com.ocs.handler.*;
import com.ocs.model.User;
import com.ocs.util.PasswordHasher;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args)
    throws Exception {
        createFirstAdminIfMissing();
        int port = Database.getInt("server.port",
        8080);
        HttpServer server =
        HttpServer.create(new
        InetSocketAddress(port), 0);

        AuthHandler auth = new
        AuthHandler();
        for (String p : new String[]{"",
        "/register", "/login", "/logout"})
        server.createContext(p, auth);

        DashboardHandler dash = new
        DashboardHandler();
        server.createContext("/dashboard",
        dash);
        server.createContext("/history", dash);

        server.createContext("/messages",
        new MessageHandler());
        server.createContext("/calendar", new
        CalendarHandler());
```

```
server.createContext("/announcements",
        new AnnouncementHandler());
        server.createContext("/polls", new
        PollHandler());
        server.createContext("/meetings", new
        MeetingHandler());
        server.createContext("/violations", new
        ViolationHandler());
        server.createContext("/admin", new
        AdminHandler());

        byte[] css =
        loadResource("/static/style.css");
        server.createContext("/style.css", ex ->
        {
            ex.getResponseHeaders().set("Content-Typ
            e", "text/css; charset=utf-8");
            ex.sendResponseHeaders(200,
            css.length);
            try (OutputStream os =
            ex.getResponseBody()) { os.write(css); }
            });

        server.setExecutor(Executors.newFixedThr
        eadPool(8));
        server.start();
        System.out.println("OCS running at
        http://localhost:" + port);
    }

    private static void
    createFirstAdminIfMissing() throws
    Exception {
        UserDao users = new UserDao();
        if (users.adminExists()) return;
        String username =
        Database.get("admin.username", "admin");
        String password =
        Database.get("admin.password",
        "ChangeMe123!");
        users.create(new
        User(Database.get("admin.name",
        "Administrator"), username,
```

```
PasswordHasher.hash(password), "admin",
"active", "");
```

```
    System.out.println("First administrator
" + username + " created. Change the
password after first login.");
}
```

```
    private static byte[] loadResource(String
path) throws IOException {
    try (InputStream in =
Main.class.getResourceAsStream(path)) {
        if (in == null) throw new
IOException("Missing resource " + path);
        return in.readAllBytes();
    }
}
```

Database.java:

```
package com.ocs;
```

```
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;
```

```
public final class Database {
    private static final Properties PROPS =
new Properties();
```

```
    static {
        File f = new
File(System.getProperty("ocs.config",
"config.properties"));
        try (InputStream in = f.exists() ? new
FileInputStream(f) :
Database.class.getResourceAsStream("/co
nfig.properties")) {
            if (in == null) {
```

```
                throw new
IllegalStateException("config.properties not
found. Copy "
```

```
                +
"src/main/resources/config.properties.exam
ple to ./config.properties and edit it.");
```

```
            }
            PROPS.load(in);
        } catch (IOException e) {
            throw new
IllegalStateException("Could not read
config.properties", e);
        }
    }
}
```

```
private Database() {}
```

```
public static String get(String key, String
def) { return PROPS.getProperty(key, def); }
```

```
public static int getInt(String key, int def) {
    try { return
Integer.parseInt(PROPS.getProperty(key,
String.valueOf(def)).trim()); }
    catch (NumberFormatException e) {
return def; }
}
```

```
public static boolean getBool(String key,
boolean def) {
    return
Boolean.parseBoolean(PROPS.getProperty
(key, String.valueOf(def)).trim());
}
```

```
public static Connection getConnection()
throws SQLException {
    return
DriverManager.getConnection(get("db.url",
""), get("db.user", ""), get("db.password",
""));
}
```

User.java:

```
package com.ocs.model;
public class User {
    public int id;
    public String name, username,
    passwordHash, role, status, department;
    public int failedAttempts;

    public User() { }
    public User(String name, String
    username, String passwordHash, String
    role, String status, String department) {
        this.name = name;
        this.username = username;
        this.passwordHash = passwordHash;
        this.role = role;
        this.status = status;
        this.department = department;
    }
    public boolean isAdmin() { return
    "admin".equals(role); }
    public boolean canManage() { return
    "manager".equals(role) ||
    "admin".equals(role); }
}
```

Message.java:

```
package com.ocs.model;
import java.sql.Timestamp;
public class Message {
    public int id, senderId, recipientId;
    public String type, body;
    public boolean anonymous;
    public Timestamp sentAt;
    public String senderName,
    recipientName;
}
```

CalendarEvent.java:

```
package com.ocs.model;
public class CalendarEvent {
    public int id;
    public String title, date, description;
}
```

Announcement.java:

```
package com.ocs.model;
import java.sql.Timestamp;
public class Announcement {
    public int id;
    public String title, body, author;
    public Timestamp postedAt;
}
```

Poll.java:

```
package com.ocs.model;
import java.util.ArrayList;
import java.util.List;

public class Poll {
    public int id;
    public String question;
    public boolean voted;
    public List<Option> options = new
    ArrayList<>();

    public static class Option {
        public int id, votes;
        public String label;
    }

    public int total() {
        int t = 0;
        for (Option o : options) t += o.votes;
        return t;
    }
}
```

Meeting.java:

```
package com.ocs.model;
import java.sql.Timestamp;
public class Meeting {
    public int id, createdById;
    public String title, invitees, createdBy;
    public Timestamp meetingTime;
}
```

```
package com.ocs.model;
import java.sql.Timestamp;
public class Violation {
    public int id;
    public String reporter, description, status;
    public Timestamp reportedAt;
}
```

```
package com.ocs.model;
import java.sql.Timestamp;
public class LoginRecord {
    public String username, ipAddress;
    public Timestamp loginTime;
}
```

```
package com.ocs.dao;

import com.ocs.Database;
import com.ocs.model.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class UserDao {
    private static final String COLS =
        "id,name,username,password_hash,role,status,department,failed_attempts";

    private User map(ResultSet rs) throws
        SQLException {
        User u = new User();
        u.id = rs.getInt("id");
        u.name = rs.getString("name");
        u.username =
            rs.getString("username");
        u.passwordHash =
            rs.getString("password_hash");
        u.role = rs.getString("role");
        u.status = rs.getString("status");
        u.department =
            rs.getString("department");
    }
}
```

```

        u.failedAttempts =
rs.getInt("failed_attempts");
        return u;
    }

    public User findByUsername(String
username) throws SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement("SELECT " + COLS + "
FROM users WHERE username=?")) {
            ps.setString(1, username);
            try (ResultSet rs =
ps.executeQuery()) { return rs.next() ?
map(rs) : null; }
        }
    }

    public User findById(int id) throws
SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement("SELECT " + COLS + "
FROM users WHERE id=?")) {
            ps.setInt(1, id);
            try (ResultSet rs =
ps.executeQuery()) { return rs.next() ?
map(rs) : null; }
        }
    }

    public boolean usernameExists(String
username) throws SQLException {
        return findByUsername(username) !=
null;
    }

    public boolean adminExists() throws
SQLException {
        try (Connection c =
Database.getConnection();
            Statement st = c.createStatement());

```

```

        ResultSet rs =
st.executeQuery("SELECT COUNT(*)
FROM users WHERE role='admin'")) {
    rs.next();
    return rs.getInt(1) > 0;
}
}

public int create(User u) throws
SQLException {
    try (Connection c =
Database.getConnection();
        PreparedStatement ps =
c.prepareStatement(
            "INSERT INTO
users(name,username,password_hash,role,
status,department) VALUES(?,?,?,?,?,?)",
Statement.RETURN_GENERATED_KEYS))
    {
        ps.setString(1, u.name);
        ps.setString(2, u.username);
        ps.setString(3, u.passwordHash);
        ps.setString(4, u.role);
        ps.setString(5, u.status);
        ps.setString(6, u.department == null
? "" : u.department);
        ps.executeUpdate();
        try (ResultSet k =
ps.getGeneratedKeys()) { return k.next() ?
k.getInt(1) : -1; }
    }
}

public List<User> findAll() throws
SQLException {
    return query("SELECT " + COLS + "
FROM users ORDER BY id");
}

public List<User> findByStatus(String
status) throws SQLException {
    List<User> list = new ArrayList<>();

```

```

        for (User u : findAll()) if
(u.status.equals(status)) list.add(u);
        return list;
    }

    public List<User> findActiveExcept(int id)
throws SQLException {
        List<User> list = new ArrayList<>();
        for (User u : findAll()) if
(u.status.equals("active") && u.id != id)
list.add(u);
        return list;
    }

    private List<User> query(String sql)
throws SQLException {
        List<User> list = new ArrayList<>();
        try (Connection c =
Database.getConnection();
            Statement st = c.createStatement();
            ResultSet rs =
st.executeQuery(sql)) {
            while (rs.next()) list.add(map(rs));
        }
        return list;
    }

    public void updateStatus(int id, String
status) throws SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement(
                "UPDATE users SET status=?,
failed_attempts=IF(=?='active',0,failed_attem
pts) WHERE id=?")) {
            ps.setString(1, status);
            ps.setString(2, status);
            ps.setInt(3, id);
            ps.executeUpdate();
        }
    }
}

```

```

    public void updateRole(int id, String role)
    throws SQLException {
        try (Connection c =
            Database.getConnection();
            PreparedStatement ps =
                c.prepareStatement("UPDATE users SET
                    role=? WHERE id=?")) {
            ps.setString(1, role);
            ps.setInt(2, id);
            ps.executeUpdate();
        }
    }

```

```

    public void recordFailedLogin(int id, int
    maxAttempts) throws SQLException {
        try (Connection c =
            Database.getConnection();
            PreparedStatement ps =
                c.prepareStatement(
                    "UPDATE users SET
                    failed_attempts=failed_attempts+1, "
                    +
                    "status=IF(failed_attempts>=?, 'locked', statu
                    s) WHERE id=? AND status='active'")) {
            ps.setInt(1, maxAttempts);
            ps.setInt(2, id);
            ps.executeUpdate();
        }
    }

```

```

    public void resetFailedLogins(int id)
    throws SQLException {
        try (Connection c =
            Database.getConnection();
            PreparedStatement ps =
                c.prepareStatement("UPDATE users SET
                    failed_attempts=0 WHERE id=?")) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}

```

MessageDao.java:

```

package com.ocs.dao;
import com.ocs.Database;
import com.ocs.model.Message;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class MessageDao {
    public void insert(Message m) throws
    SQLException {
        try (Connection c =
            Database.getConnection();
            PreparedStatement ps =
                c.prepareStatement(
                    "INSERT INTO
                    messages(sender_id,recipient_id,type,body,
                    anonymous) VALUES(?,?,?,?,?)")) {
            ps.setInt(1, m.senderId);
            ps.setInt(2, m.recipientId);
            ps.setString(3, m.type);
            ps.setString(4, m.body);
            ps.setBoolean(5, m.anonymous);
            ps.executeUpdate();
        }
    }

    public List<Message> inbox(int userId)
    throws SQLException {
        String sql = "SELECT m.id, m.type,
            m.body, m.anonymous, m.sent_at, "
            + "CASE WHEN m.anonymous
            THEN 'Anonymous' ELSE u.name END AS
            sender_name "
            + "FROM messages m JOIN
            users u ON u.id = m.sender_id "
            + "WHERE m.recipient_id=?
            ORDER BY m.sent_at DESC, m.id DESC";
        List<Message> list = new
        ArrayList<>();
        try (Connection c =
            Database.getConnection();
            PreparedStatement ps =
                c.prepareStatement(sql)) {

```

```

        ps.setInt(1, userId);
        try (ResultSet rs =
ps.executeQuery()) {
            while (rs.next()) {
                Message m = new Message();
                m.id = rs.getInt("id");
                m.type = rs.getString("type");
                m.body = rs.getString("body");
                m.anonymous =
rs.getBoolean("anonymous");
                m.sentAt =
rs.getTimestamp("sent_at");
                m.senderName =
rs.getString("sender_name");
                list.add(m);
            }
        }
    }
    return list;
}

```

```

    public List<Message> sent(int userId)
throws SQLException {
        String sql = "SELECT m.id, m.type,
m.body, m.anonymous, m.sent_at, u.name
AS recipient_name "
+ "FROM messages m JOIN
users u ON u.id = m.recipient_id "
+ "WHERE m.sender_id=?
ORDER BY m.sent_at DESC, m.id DESC";
        List<Message> list = new
ArrayList<>();
        try (Connection c =
Database.getConnection();
PreparedStatement ps =
c.prepareStatement(sql)) {
            ps.setInt(1, userId);
            try (ResultSet rs =
ps.executeQuery()) {
                while (rs.next()) {
                    Message m = new Message();
                    m.id = rs.getInt("id");
                    m.type = rs.getString("type");
                    m.body = rs.getString("body");

```

```

                    m.anonymous =
rs.getBoolean("anonymous");
                    m.sentAt =
rs.getTimestamp("sent_at");
                    m.recipientName =
rs.getString("recipient_name");
                    list.add(m);
                }
            }
        }
        return list;
    }
}

```

CalendarDao.java:

```

package com.ocs.dao;
import com.ocs.Database;
import com.ocs.model.CalendarEvent;
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class CalendarDao {
    public void insert(String title, LocalDate
date, String description, int createdBy)
throws SQLException {
        try (Connection c =
Database.getConnection();
PreparedStatement ps =
c.prepareStatement(
"INSERT INTO
calendar_events(title,event_date,description
,created_by) VALUES(?,?,?,?)")) {
            ps.setString(1, title);
            ps.setDate(2, Date.valueOf(date));
            ps.setString(3, description);
            ps.setInt(4, createdBy);
            ps.executeUpdate();
        }
    }

    public List<CalendarEvent> listAll()
throws SQLException {

```

```

        List<CalendarEvent> list = new
        ArrayList<>();
        try (Connection c =
        Database.getConnection();
            Statement st = c.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT
id,title,event_date,description FROM
calendar_events ORDER BY event_date,
id")) {
            while (rs.next()) {
                CalendarEvent e = new
                CalendarEvent();
                e.id = rs.getInt("id");
                e.title = rs.getString("title");
                e.date =
                rs.getString("event_date");
                e.description =
                rs.getString("description");
                list.add(e);
            }
        }
        return list;
    }
}

```

```

    public void delete(int id) throws
    SQLException {
        try (Connection c =
        Database.getConnection();
            PreparedStatement ps =
            c.prepareStatement("DELETE FROM
calendar_events WHERE id=?")) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}

```

AnnouncementDao.java:

```

package com.ocs.dao;
import com.ocs.Database;
import com.ocs.model.Announcement;
import java.sql.*;
import java.util.ArrayList;

```

```

import java.util.List;

public class AnnouncementDao {
    public void insert(String title, String body,
    int createdBy) throws SQLException {
        try (Connection c =
        Database.getConnection();
            PreparedStatement ps =
            c.prepareStatement(
                "INSERT INTO
announcements(title,body,created_by)
VALUES(?,?,?)")) {
            ps.setString(1, title);
            ps.setString(2, body);
            ps.setInt(3, createdBy);
            ps.executeUpdate();
        }
    }

    public List<Announcement> listAll()
    throws SQLException {
        List<Announcement> list = new
        ArrayList<>();
        try (Connection c =
        Database.getConnection();
            Statement st = c.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT
a.id,a.title,a.body,a.posted_at,u.name AS
author FROM announcements a "
                + "LEFT JOIN users u ON
u.id=a.created_by ORDER BY a.posted_at
DESC, a.id DESC")) {
            while (rs.next()) {
                Announcement a = new
                Announcement();
                a.id = rs.getInt("id");
                a.title = rs.getString("title");
                a.body = rs.getString("body");
                a.postedAt =
                rs.getTimestamp("posted_at");
                a.author = rs.getString("author");
                list.add(a);
            }
        }
    }
}

```



```

                o.id = rs.getInt("id");
                o.label =
rs.getString("label");
                o.votes =
rs.getInt("votes");
                p.options.add(o);
            }
        }
    }
    try (PreparedStatement ps =
c.prepareStatement(
        "SELECT 1 FROM
poll_votes WHERE poll_id=? AND
user_id=?")) {
        ps.setInt(1, p.id);
        ps.setInt(2, userId);
        try (ResultSet rs =
ps.executeQuery()) { p.voted = rs.next(); }
    }
}
return polls;
}

    public boolean vote(int pollId, int userId,
int optionId) throws SQLException {
    try (Connection c =
Database.getConnection()) {
        try (PreparedStatement ps =
c.prepareStatement("SELECT 1 FROM
poll_options WHERE id=? AND poll_id=?"))
        {
            ps.setInt(1, optionId);
            ps.setInt(2, pollId);
            try (ResultSet rs =
ps.executeQuery()) { if (!rs.next()) return
false; }
        }
        try (PreparedStatement ps =
c.prepareStatement(
            "INSERT INTO
poll_votes(poll_id,user_id,option_id)
VALUES(?,?,?)")) {
            ps.setInt(1, pollId);

```

```

                ps.setInt(2, userId);
                ps.setInt(3, optionId);
                ps.executeUpdate();
                return true;
            } catch
(SQLIntegrityConstraintViolationException
duplicate) {
                return false;
            }
        }
    }
}

```

MeetingDao.java:

```

package com.ocs.dao;
import com.ocs.Database;
import com.ocs.model.Meeting;
import java.sql.*;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class MeetingDao {
    public static Set<String> split(String csv) {
        Set<String> set = new
LinkedHashSet<>();
        if (csv == null) return set;
        for (String s : csv.split(",")) {
            String t = s.trim();
            if (!t.isEmpty()) set.add(t);
        }
        return set;
    }

    public void insert(String title, Timestamp
time, String invitees, int createdBy) throws
SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement(

```

```

        "INSERT INTO
meetings(title,meeting_time,invitees,created
_by) VALUES(?,?,?,?))" {
    ps.setString(1, title);
    ps.setTimestamp(2, time);
    ps.setString(3, invitees);
    ps.setInt(4, createdBy);
    ps.executeUpdate();
    }
}

public List<Meeting> listAll() throws
SQLException {
    List<Meeting> list = new ArrayList<>();
    try (Connection c =
Database.getConnection();
        Statement st = c.createStatement();
        ResultSet rs = st.executeQuery(
            "SELECT
m.id,m.title,m.meeting_time,m.invitees,m.cr
eated_by,u.username "
            + "FROM meetings m
JOIN users u ON u.id=m.created_by
ORDER BY m.meeting_time")) {
        while (rs.next()) {
            Meeting m = new Meeting();
            m.id = rs.getInt("id");
            m.title = rs.getString("title");
            m.meetingTime =
rs.getTimestamp("meeting_time");
            m.invitees =
rs.getString("invitees");
            m.createdById =
rs.getInt("created_by");
            m.createdBy =
rs.getString("username");
            list.add(m);
        }
    }
    return list;
}

```

```

    public boolean hasConflict(Timestamp
time, int creatorId, Set<String> invitees)
throws SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement(
                "SELECT invitees, created_by
FROM meetings WHERE
meeting_time=?") {
            ps.setTimestamp(1, time);
            try (ResultSet rs =
ps.executeQuery()) {
                while (rs.next()) {
                    if (rs.getInt("created_by") ==
creatorId) return true;
                    for (String inv :
split(rs.getString("invitees"))) {
                        if (invitees.contains(inv))
return true;
                    }
                }
            }
        }
        return false;
    }
}

```

ViolationDao.java:

```

package com.ocs.dao;
import com.ocs.Database;
import com.ocs.model.Violation;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ViolationDao {
    public void insert(int reporterId, String
description) throws SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement("INSERT INTO

```

```

violations(reporter_id,description)
VALUES(?,?)) {
    ps.setInt(1, reporterId);
    ps.setString(2, description);
    ps.executeUpdate();
}
}

```

```

public List<Violation> listAll() throws
SQLException {
    List<Violation> list = new ArrayList<>();
    try (Connection c =
Database.getConnection();
        Statement st = c.createStatement();
        ResultSet rs = st.executeQuery(
            "SELECT
v.id,v.description,v.status,v.reported_at,u.us
ername FROM violations v "
            + "JOIN users u ON
u.id=v.reporter_id ORDER BY v.status,
v.reported_at DESC")) {
        while (rs.next()) {
            Violation v = new Violation();
            v.id = rs.getInt("id");
            v.description =
rs.getString("description");
            v.status = rs.getString("status");
            v.reportedAt =
rs.getTimestamp("reported_at");
            v.reporter =
rs.getString("username");
            list.add(v);
        }
    }
    return list;
}

```

```

public void resolve(int id) throws
SQLException {
    try (Connection c =
Database.getConnection();
        PreparedStatement ps =
c.prepareStatement("UPDATE violations
SET status='resolved' WHERE id=?")) {

```

```

        ps.setInt(1, id);
        ps.executeUpdate();
    }
}

```

LoginHistoryDao.java:

```

package com.ocs.dao;
import com.ocs.Database;
import com.ocs.model.LoginRecord;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class LoginHistoryDao {
    public void record(int userId, String ip)
throws SQLException {
        try (Connection c =
Database.getConnection();
            PreparedStatement ps =
c.prepareStatement("INSERT INTO
login_history(user_id,ip_address)
VALUES(?,?)")) {
            ps.setInt(1, userId);
            ps.setString(2, ip);
            ps.executeUpdate();
        }
    }

    public List<LoginRecord> forUser(int
userId) throws SQLException {
        return query("SELECT
u.username,h.login_time,h.ip_address
FROM login_history h "
            + "JOIN users u ON
u.id=h.user_id WHERE h.user_id=?
ORDER BY h.login_time DESC, h.id DESC
LIMIT 200", userId);
    }

    public List<LoginRecord> all() throws
SQLException {

```

```

        return query("SELECT
u.username,h.login_time,h.ip_address
FROM login_history h "
        + "JOIN users u ON
u.id=h.user_id ORDER BY h.login_time
DESC, h.id DESC LIMIT 500", null);
    }

```

```

    private List<LoginRecord> query(String
sql, Integer param) throws SQLException {
        List<LoginRecord> list = new
ArrayList<>();
        try (Connection c =
Database.getConnection();
PreparedStatement ps =
c.prepareStatement(sql)) {
            if (param != null) ps.setInt(1, param);
            try (ResultSet rs =
ps.executeQuery()) {
                while (rs.next()) {
                    LoginRecord r = new
LoginRecord();
                    r.username =
rs.getString("username");
                    r.loginTime =
rs.getTimestamp("login_time");
                    r.ipAddress =
rs.getString("ip_address");
                    list.add(r);
                }
            }
        }
        return list;
    }

```

```

    public void logAdminAction(int adminId,
String action, String details) throws
SQLException {
        try (Connection c =
Database.getConnection();
PreparedStatement ps =
c.prepareStatement("INSERT INTO
audit_log(admin_id,action,details)
VALUES(?,?,?)")) {

```

```

            ps.setInt(1, adminId);
            ps.setString(2, action);
            ps.setString(3, details);
            ps.executeUpdate();
        }
    }

```

```

    public List<String[]>
recentAdminActions(int limit) throws
SQLException {
        List<String[]> rows = new
ArrayList<>();
        try (Connection c =
Database.getConnection();
PreparedStatement ps =
c.prepareStatement(
            "SELECT
a.created_at,u.username,a.action,a.details
FROM audit_log a "
            + "JOIN users u ON
u.id=a.admin_id ORDER BY a.created_at
DESC, a.id DESC LIMIT ?")) {
            ps.setInt(1, limit);
            try (ResultSet rs =
ps.executeQuery()) {
                while (rs.next()) {
                    rows.add(new
String[]{String.valueOf(rs.getTimestamp(1)),
rs.getString(2), rs.getString(3),
rs.getString(4)});
                }
            }
        }
        return rows;
    }

```

BaseHandler.java:

```

package com.ocs.handler;
import com.ocs.Database;
import com.ocs.dao.UserDao;
import com.ocs.model.User;
import com.ocs.util.HtmlUtil;
import com.ocs.util.SessionManager;

```

```

import
com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import java.io.IOException;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.util.List;

public abstract class BaseHandler
implements HttpHandler {
    protected static final UserDao USERS =
new UserDao();
    protected abstract void
route(HttpExchange ex) throws Exception;

    @Override
    public final void handle(HttpExchange ex)
throws IOException {
        try {
            route(ex);
        } catch (Exception e) {
            e.printStackTrace();
        } try {
            sendHtml(ex, 500,
HtmlUtil.page("Error", null,
                HtmlUtil.error("The system
encountered a problem. Please try again
shortly.")));
        } catch (IOException |
IllegalStateException ignored) { }
        } finally {
            ex.close();
        }
    }

    protected static String path(HttpExchange
ex) { return ex.getRequestURI().getPath(); }
    protected static boolean
isPost(HttpExchange ex) { return
"POST".equals(ex.getRequestMethod()); }

    protected static String
cookie(HttpExchange ex, String name) {

```

```

List<String> headers =
ex.getRequestHeaders().get("Cookie");
if (headers == null) return null;
for (String h : headers) {
    for (String part : h.split(";")) {
        String[] kv = part.trim().split("=",
2);
        if (kv.length == 2 &&
kv[0].equals(name)) return kv[1];
    }
}
return null;

    protected static String
clientIp(HttpExchange ex) {
        return
ex.getRemoteAddress().getAddress().getHo
stAddress();
    }

    protected static void
setSessionCookie(HttpExchange ex, String
token, boolean clear) {
        String v = "session=" + (clear ? "" :
token) + "; Path=/; HttpOnly; SameSite=Lax"
+ (clear ? "; Max-Age=0" : "")
+
(Database.getBool("cookie.secure", false) ?
"; Secure" : "");

ex.getResponseHeaders().add("Set-Cookie"
, v);
    }

    protected static User
currentUser(HttpExchange ex) throws
Exception {
        Integer id =
SessionManager.getUserId(cookie(ex,
"session"));
        if (id == null) return null;
        User u = USERS.findById(id);

```

```

        return (u != null &&
u.status.equals("active")) ? u : null;
    }

```

```

    protected static User
requireLogin(HttpExchange ex) throws
Exception {
    User u = currentUser(ex);
    if (u == null) redirect(ex, "/");
    return u;
}

```

```

    protected static User
requireRole(HttpExchange ex, String...
roles) throws Exception {
    User u = requireLogin(ex);
    if (u == null) return null;
    for (String r : roles) if (r.equals(u.role))
return u;
    sendHtml(ex, 403,
HtmlUtil.page("Forbidden", u,
HtmlUtil.error("You do not have permission
to view this page.")));
    return null;
}

```

```

    protected static void
redirect(HttpExchange ex, String location)
throws IOException {
ex.getResponseHeaders().set("Location",
location);
    ex.sendResponseHeaders(302, -1);
}

```

```

    protected static void
sendHtml(HttpExchange ex, int status,
String html) throws IOException {
    byte[] bytes =
html.getBytes(StandardCharsets.UTF_8);

ex.getResponseHeaders().set("Content-Typ
e", "text/html; charset=utf-8");
    ex.sendResponseHeaders(status,
bytes.length);
}

```

```

        try (OutputStream os =
ex.getResponseBody()) { os.write(bytes); }
    }

```

```

    protected static void
notFound(HttpExchange ex, User u) throws
IOException {
    sendHtml(ex, 404, HtmlUtil.page("Not
found", u, "<p>Page not found. <a
href='/'>Go home</a></p>"));
}
}

```

AuthHandler.java:

```

package com.ocs.handler;
import com.ocs.Database;
import com.ocs.dao.LoginHistoryDao;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import com.ocs.util.PasswordHasher;
import com.ocs.util.SessionManager;
import
com.sun.net.httpserver.HttpExchange;
import java.util.Map;

```

```

public class AuthHandler extends
BaseHandler {
    private final LoginHistoryDao history =
new LoginHistoryDao();
}

```

```

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        switch (path(ex)) {
            case "/": home(ex); break;
            case "/register": register(ex); break;
            case "/login": login(ex); break;
            case "/logout": logout(ex); break;
            default: notFound(ex, null);
        }
    }
}

```

```

private void home(HttpExchange ex)
throws Exception {
    if (currentUser(ex) != null) { redirect(ex,
"/dashboard"); return; }
    String flash =
HtmlUtil.flash(FormParser.query(ex).get("msg"),
    "registered", "Registration
submitted. An administrator must approve
your account before you can log in.",
    "loggedout", "You have been
logged out.",
    "!badlogin", "Invalid username or
password.",
    "!pending", "Your registration is
still awaiting administrator approval.",
    "!rejected", "Your registration was
rejected. Please contact an administrator.",
    "!suspended", "Your account is
suspended. Please contact an
administrator.",
    "!deactivated", "Your account has
been deactivated.",
    "!locked", "Your account is locked
after too many failed logins. Please contact
an administrator.",
    "!exists", "That username is
already taken.",
    "!invalid", "Please fill in all fields.
Username: 3-30 letters/digits/._- Password:
at least 8 characters.");
    String body = flash + "<div
class='row'>"
        + "<div
class='card'><h2>Login</h2><form
method='POST' action='/login'>"
        +
"<label>Username</label><input type='text'
name='username' required>"
        + "<label>Password</label><input
type='password' name='password'
required>"
        + "<button
type='submit'>Login</button></form></div>"

```

```

        + "<div
class='card'><h2>Register</h2><form
method='POST' action='/register'>"
        + "<label>Full name</label><input
type='text' name='name' required>"
        +
"<label>Username</label><input type='text'
name='username' required>"
        +
"<label>Department</label><input
type='text' name='department'>"
        + "<label>Password</label><input
type='password' name='password'
required>"
        + "<label>Role</label><select
name='role'><option
value='employee'>Employee</option>"
        + "<option
value='manager'>Manager</option></select
>"
        + "<button
type='submit'>Register</button></form></div>";
    sendHtml(ex, 200,
HtmlUtil.page("Organizational
Communication System", null, body));
}

private void register(HttpExchange ex)
throws Exception {
    if (!isPost(ex)) { redirect(ex, "/"); return;
}
    Map<String, String> f =
FormParser.form(ex);
    String name = f.getOrDefault("name",
"" ).trim();
    String username =
f.getOrDefault("username", "").trim();
    String department =
f.getOrDefault("department", "").trim();
    String password =
f.getOrDefault("password", "");
    String role = f.getOrDefault("role",
"employee");

```

```

        if (!role.equals("manager")) role =
"employee";

        if (name.isEmpty() ||
!username.matches("[A-Za-z0-9_-]{3,30}")
|| password.length() < 8) {
            redirect(ex, "?msg=invalid"); return;
        }
        if (USERS.usernameExists(username))
{ redirect(ex, "?msg=exists"); return; }
        USERS.create(new User(name,
username,
PasswordHasher.hash(password), role,
"pending", department));
        redirect(ex, "?msg=registered");
    }

```

```

private void login(HttpExchange ex)
throws Exception {
    if (!isPost(ex)) { redirect(ex, "/"); return;
}
    Map<String, String> f =
FormParser.form(ex);
    User u =
USERS.findByUsername(f.getDefault("us
ername", "").trim());
    String password =
f.getDefault("password", "");

    if (u == null) { redirect(ex,
"?msg=badlogin"); return; }
    if (u.status.equals("locked")) {
redirect(ex, "?msg=locked"); return; }

```

```

        if (!PasswordHasher.verify(password,
u.passwordHash)) {
            USERS.recordFailedLogin(u.id,
Database.getInt("login.max.attempts", 5));
            redirect(ex, "?msg=badlogin");
            return;
        }
        if (!u.status.equals("active")) {
            redirect(ex, "?msg=" + u.status);
            return;

```

```

        }
        USERS.resetFailedLogins(u.id);
        history.record(u.id, clientIp(ex));
        setSessionCookie(ex,
SessionManager.create(u.id), false);
        redirect(ex, "/dashboard");
    }

    private void logout(HttpExchange ex)
throws Exception {
        SessionManager.destroy(cookie(ex,
"session"));
        setSessionCookie(ex, "", true);
        redirect(ex, "?msg=loggedout");
    }
}

```

DashboardHandler.java:

```

package com.ocs.handler;

import com.ocs.dao.LoginHistoryDao;
import com.ocs.model.LoginRecord;
import com.ocs.model.User;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;

import java.util.List;

public class DashboardHandler extends
BaseHandler {
    private final LoginHistoryDao history =
new LoginHistoryDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        User u = requireLogin(ex);
        if (u == null) return;
        if (path(ex).equals("/history"))
showHistory(ex, u); else dashboard(ex, u);
    }
}

```

```

    private static String card(String title,
String text, String href) {
    return "<div class='card'><h2>" + title +
"</h2>" + (text.isEmpty() ? "" : "<p>" + text +
"</p>")
        + "<a class='btn' href='" + href +
"">Open</a></div>";
    }

```

```

    private void dashboard(HttpExchange ex,
User u) throws Exception {
        StringBuilder b = new
StringBuilder("<h1>Welcome, " +
HtmlUtil.esc(u.name) + "</h1>");
        b.append("<div class='row'>")
            .append(card("Messages", "Send
and view messages, complaints, or
feedback.", "/messages"))
            .append(card("Calendar", "View
company and department events.",
"/calendar")).append("</div>");
        b.append("<div class='row'>")
            .append(card("Announcements",
"", "/announcements"))
            .append(card("Polls", "", "/polls"))
            .append(card("Meetings", "",
"/meetings"))
            .append(card("My Login History",
"See your past login activity.",
"/history")).append("</div>");
        if (u.isAdmin()) {
            b.append("<div class='row'>")
                .append(card("User Validation
& Access", "", "/admin/users"))
                .append(card("Violations", "",
"/admin/violations"))
                .append(card("Login &
Audit Logs", "",
"/admin/history")).append("</div>");
        } else {
            b.append(card("Report an Issue", "",
"/violations/report"));
        }
    }

```

```

        sendHtml(ex, 200,
HtmlUtil.page("Dashboard", u, b.toString()));
    }

```

```

    private void showHistory(HttpExchange
ex, User u) throws Exception {
        List<LoginRecord> rows =
history.forUser(u.id);
        StringBuilder b = new
StringBuilder("<h1>My Login
History</h1><div
class='card'><table><tr><th>Time</th><th>I
P address</th></tr>");
        for (LoginRecord r : rows) {
            b.append("<tr><td>").append(HtmlUtil.fmt(r.l
oginTime)).append("</td><td>")
                .append(HtmlUtil.esc(r.ipAddress)).append("
</td></tr>");
        }
        b.append("</table></div>");
        sendHtml(ex, 200,
HtmlUtil.page("Login History", u,
b.toString()));
    }
}

```

MessageHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.MessageDao;
import com.ocs.model.Message;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;
import java.util.Map;
import java.util.Set;

public class MessageHandler extends
BaseHandler {
    private static final Set<String> TYPES =
Set.of("feedback", "complaint", "request",
"general");

```

```

private final MessageDao messages =
new MessageDao();

@Override
protected void route(HttpExchange ex)
throws Exception {
    User u = requireLogin(ex);
    if (u == null) return;
    switch (path(ex)) {
        case "/messages": show(ex, u);
        break;
        case "/messages/send": send(ex, u);
        break;
        default: notFound(ex, u);
    }
}

private void show(HttpExchange ex, User
u) throws Exception {
    StringBuilder b = new
StringBuilder("<h1>Messages</h1>");

b.append(HtmlUtil.flash(FormParser.query(e
x).get("msg"),
    "sent", "Message sent.",
    "invalid", "Choose a recipient and
type a message before sending."));
    b.append("<div
class='card'><h2>Compose</h2><form
method='POST'
action='/messages/send'>");
    b.append("<label>To</label><select
name='to'>");
    for (User o :
USERS.findActiveExcept(u.id)) {
        b.append("<option
value=''>").append(o.id).append("<option>");
        b.append(HtmlUtil.esc(o.name))
        .append("
").append(HtmlUtil.esc(o.role)).append("</
option>");
    }
    b.append("</select><label>Type</label><se
lect name='type'>")

```

```

        .append("<option
value='general'>General
message</option><option
value='feedback'>Feedback</option>")
        .append("<option
value='complaint'>Complaint</option><optio
n
value='request'>Request</option></select>")
    );

b.append("<label>Message</label><textare
a name='body' rows='3'
required></textarea>");
    b.append("<label><input
type='checkbox' name='anonymous'
style='width:auto;display:inline-block;'>
Send anonymously</label>");
    b.append("<p
class='muted'>Anonymity cannot be
changed after the message is sent.</p>");
    b.append("<button
type='submit'>Send</button></form></div>")
    );

    b.append("<div class='row'><div
class='card'><h2>Inbox</h2>");
    java.util.List<Message> inbox =
messages.inbox(u.id);
    if (inbox.isEmpty()) b.append("<p>No
messages received.</p>");
    for (Message m : inbox) {
        b.append("<div
class='msg'><b>").append(HtmlUtil.esc(m.s
enderName)).append("</b>")
        .append("<span
class='tag'>").append(HtmlUtil.esc(m.type)).
append("</span> &mdash; ")
        .append(HtmlUtil.esc(m.body)).append("<br
><span class='muted'>")

        .append(HtmlUtil.fmt(m.sentAt)).append("</
span></div>");
    }
}

```

```

        b.append("</div><div
class='card'><h2>Sent</h2>");
        java.util.List<Message> sent =
messages.sent(u.id);
        if (sent.isEmpty()) b.append("<p>No
messages sent.</p>");
        for (Message m : sent) {
            b.append("<div class='msg'>To
<b>").append(HtmlUtil.esc(m.recipientName
)).append("</b>")
                .append(m.anonymous ? "
(sent anonymously)" : "")
                .append("<span
class='tag'>").append(HtmlUtil.esc(m.type)).
append("</span> &mdash; ")

.append(HtmlUtil.esc(m.body)).append("<br
><span class='muted'>")

.append(HtmlUtil.fmt(m.sentAt)).append("</
span></div>");
        }
        b.append("</div></div>");
        sendHtml(ex, 200,
HtmlUtil.page("Messages", u, b.toString()));
    }

    private void send(HttpExchange ex, User
u) throws Exception {
        if (!isPost(ex)) { redirect(ex,
"/messages"); return; }
        Map<String, String> f =
FormParser.form(ex);
        int to = FormParser.toInt(f.get("to"), -1);
        String body = f.getDefault("body",
"" ).trim();
        String type = f.getDefault("type",
"general");
        User recipient = USERS.findById(to);

        if (recipient == null ||
!recipient.status.equals("active") ||
recipient.id == u.id

```

```

        || body.isEmpty() ||
!TYPES.contains(type)) {
            redirect(ex,
"/messages?msg=invalid");
            return;
        }
        Message m = new Message();
        m.senderId = u.id;
        m.recipientId = recipient.id;
        m.type = type;
        m.body = body;
        m.anonymous =
f.containsKey("anonymous");
        messages.insert(m);
        redirect(ex, "/messages?msg=sent");
    }
}

```

CalendarHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.CalendarDao;
import com.ocs.model.CalendarEvent;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;
import java.time.LocalDate;
import java.util.Map;

public class CalendarHandler extends
BaseHandler {
    private final CalendarDao calendar = new
CalendarDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        switch (path(ex)) {
            case "/calendar": {
                User u = requireLogin(ex);
                if (u != null) show(ex, u);
                break;
            }
        }
    }
}

```

```

        case "/calendar/create": {
            User u = requireRole(ex,
"manager", "admin");
            if (u != null) create(ex, u);
            break;
        }
        case "/calendar/delete": {
            User u = requireRole(ex,
"manager", "admin");
            if (u != null) delete(ex);
            break;
        }
        default: notFound(ex, null);
    }
}

private void show(HttpExchange ex, User
u) throws Exception {
    StringBuilder b = new
StringBuilder("<h1>Calendar</h1>");

    b.append(HtmlUtil.flash(FormParser.query(e
x).get("msg"), "!invalid", "Please enter a title
and a valid date."));
    if (u.canManage()) {
        b.append("<div
class='card'><h2>Add Event</h2><form
method='POST' action='/calendar/create'>")

        .append("<label>Title</label><input
type='text' name='title' required>")

        .append("<label>Date</label><input
type='date' name='date' required>")

        .append("<label>Description</label><textar
ea name='description'
rows='2'></textarea>")
        .append("<button
type='submit'>Add</button></form></div>");
    }
    b.append("<div
class='card'><h2>Upcoming

```

```

Events</h2><table><tr><th>Date</th><th>
Title</th><th>Description</th>");
        if (u.canManage())
            b.append("<th></th>");
            b.append("</tr>");
            for (CalendarEvent e :
calendar.listAll()) {

                b.append("<tr><td>").append(HtmlUtil.esc(e
.date)).append("</td><td>").append(HtmlUtil
.esc(e.title))

                .append("</td><td>").append(HtmlUtil.esc(e
.description)).append("</td>");
                if (u.canManage()) {

                    b.append("<td>").append(HtmlUtil.postForm
("/calendar/delete", "Delete", "btn-small
btn-danger",

                        "id",
String.valueOf(e.id))).append("</td>");
                }
                b.append("</tr>");
            }
            b.append("</table></div>");
            sendHtml(ex, 200,
HtmlUtil.page("Calendar", u, b.toString()));
        }

        private void create(HttpExchange ex,
User u) throws Exception {
            if (!isPost(ex)) { redirect(ex,
"/calendar"); return; }
            Map<String, String> f =
FormParser.form(ex);
            String title = f.getDefault("title",
"").trim();
            try {
                LocalDate date =
LocalDate.parse(f.getDefault("date", ""));
                if (title.isEmpty()) throw new
IllegalArgumentException();
                calendar.insert(title, date,
f.getDefault("description", "").trim(), u.id);

```

```

        redirect(ex, "/calendar");
    } catch (RuntimeException e) {
        redirect(ex,
"/calendar?msg=invalid");
    }
}

private void delete(HttpExchange ex)
throws Exception {
    if (isPost(ex))
calendar.delete(FormParser.toInt(FormParser.form(ex).get("id"), -1));
        redirect(ex, "/calendar");
    }
}

```

AnnouncementHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.AnnouncementDao;
import com.ocs.model.Announcement;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;
import java.util.Map;

public class AnnouncementHandler extends
BaseHandler {
    private final AnnouncementDao
announcements = new
AnnouncementDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        switch (path(ex)) {
            case "/announcements": {
                User u = requireLogin(ex);
                if (u != null) show(ex, u);
                break;
            }
            case "/announcements/create": {

```

```

                User u = requireRole(ex,
"manager", "admin");
                if (u != null) create(ex, u);
                break;
            }
            default: notFound(ex, null);
        }
    }

    private void show(HttpExchange ex, User
u) throws Exception {
        StringBuilder b = new
StringBuilder("<h1>Announcements</h1>");

        b.append(HtmlUtil.flash(FormParser.query(e
x).get("msg"), "!invalid", "Title and body are
required."));
        if (u.canManage()) {
            b.append("<div
class='card'><h2>New
Announcement</h2><form method='POST'
action='/announcements/create'>")

            .append("<label>Title</label><input
type='text' name='title' required>")

            .append("<label>Body</label><textarea
name='body' rows='3'
required></textarea>")
                .append("<button
type='submit'>Post</button></form></div>")
            ;
        }
        b.append("<div
class='card'><h2>Feed</h2>");
        java.util.List<Announcement> list =
announcements.listAll();
        if (list.isEmpty()) b.append("<p>No
announcements yet.</p>");
        for (Announcement a : list) {
            b.append("<div
class='msg'><b>").append(HtmlUtil.esc(a.titl
e)).append("</b><br>")

```

```

.append(HtmlUtil.esc(a.body)).append("<br>
<span class='muted'>")
        .append(HtmlUtil.esc(a.author
== null ? "System" : a.author)).append("
&middot; ")
.append(HtmlUtil.fmt(a.postedAt)).append("
</span></div>");
    }
    b.append("</div>");
    sendHtml(ex, 200,
HtmlUtil.page("Announcements", u,
b.toString()));
}

private void create(HttpExchange ex,
User u) throws Exception {
    if (!isPost(ex)) { redirect(ex,
"/announcements"); return; }
    Map<String, String> f =
FormParser.form(ex);
    String title = f.getDefault("title",
"" ).trim();
    String body = f.getDefault("body",
"" ).trim();
    if (title.isEmpty() || body.isEmpty()) {
redirect(ex,
"/announcements?msg=invalid"); return; }
    announcements.insert(title, body, u.id);
    redirect(ex, "/announcements");
}
}

```

PollHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.PollDao;
import com.ocs.model.Poll;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;
import java.util.ArrayList;
import java.util.List;

```

```

import java.util.Map;

```

```

public class PollHandler extends
BaseHandler {
    private final PollDao polls = new
PollDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        switch (path(ex)) {
            case "/polls": {
                User u = requireLogin(ex);
                if (u != null) show(ex, u);
                break;
            }
            case "/polls/create": {
                User u = requireRole(ex,
"manager", "admin");
                if (u != null) create(ex, u);
                break;
            }
            case "/polls/vote": {
                User u = requireLogin(ex);
                if (u != null) vote(ex, u);
                break;
            }
            default: notFound(ex, null);
        }
    }
}

```

```

    private void show(HttpExchange ex, User
u) throws Exception {
        StringBuilder b = new
StringBuilder("<h1>Polls</h1>");
        b.append(HtmlUtil.flash(FormParser.query(e
x).get("msg"),
            "!invalid", "A poll needs a question
and between 2 and 10 options.",
            "!novote", "Your vote could not be
recorded (already voted or invalid
option)."));
        if (u.canManage()) {

```

```

        b.append("<div
class='card'><h2>New Poll</h2><form
method='POST' action='/polls/create'>")
        .append("<label>Question</label><input
type='text' name='question' required>")
        .append("<label>Options
(comma-separated)</label>")
        .append("<input type='text'
name='options' placeholder='Option A,
Option B, Option C' required>")
        .append("<button
type='submit'>Create</button></form></div
>");
    }
    List<Poll> list = polls.listAll(u.id);
    for (Poll p : list) {
        b.append("<div
class='card'><h2>").append(HtmlUtil.esc(p.
question)).append("</h2>");
        if (!p.voted) {
            b.append("<form method='POST'
action='/polls/vote'><input type='hidden'
name='pollId' value='")
                .append(p.id).append("'>");
            for (Poll.Option o : p.options) {
                b.append("<label><input
type='radio' name='option'
value='").append(o.id)
                    .append("' required
style='width:auto;display:inline-block;'> ")
                    .append(HtmlUtil.esc(o.label)).append("</la
bel><br>");
            }
            b.append("<br><button
type='submit'
class='btn-small'>Vote</button></form>");
        } else {
            b.append("<p><i>You already
voted.</i></p>");
        }
        b.append("<table><tr><th>Option</th><th>
Votes</th></tr>");
        for (Poll.Option o : p.options) {

```

```

            b.append("<tr><td>").append(HtmlUtil.esc(o
.label)).append("</td><td>").append(o.votes
).append("</td></tr>");
        }
        b.append("</table><p>Total votes:
").append(p.total()).append("</p></div>");
    }
    if (list.isEmpty()) b.append("<div
class='card'><p>No polls yet.</p></div>");
    sendHtml(ex, 200,
HtmlUtil.page("Polls", u, b.toString()));
}

private void create(HttpExchange ex,
User u) throws Exception {
    if (!isPost(ex)) { redirect(ex, "/polls");
return; }
    Map<String, String> f =
FormParser.form(ex);
    String question =
f.getDefault("question", "").trim();
    List<String> options = new
ArrayList<>();
    for (String o : f.getDefault("options",
"" ).split(",")) {
        String t = o.trim();
        if (!t.isEmpty() &&
!options.contains(t)) options.add(t);
    }
    if (question.isEmpty() || options.size() <
2 || options.size() > 10) {
        redirect(ex, "/polls?msg=invalid");
return;
    }
    polls.create(question, options, u.id);
    redirect(ex, "/polls");
}

```

```

private void vote(HttpExchange ex, User
u) throws Exception {
    if (!isPost(ex)) { redirect(ex, "/polls");
return; }

```

```

        Map<String, String> f =
FormParser.form(ex);
        boolean ok =
polls.vote(FormParser.toInt(f.get("pollId"),
-1), u.id, FormParser.toInt(f.get("option"),
-1));
        redirect(ex, ok ? "/polls" :
"/polls?msg=novote");
    }
}

```

MeetingHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.MeetingDao;
import com.ocs.model.Meeting;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;
import java.sql.Timestamp;
import java.time.LocalDateTime;
import java.util.Map;
import java.util.Set;

public class MeetingHandler extends
BaseHandler {
    private final MeetingDao meetings = new
MeetingDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        switch (path(ex)) {
            case "/meetings": {
                User u = requireLogin(ex);
                if (u != null) show(ex, u);
                break;
            }
            case "/meetings/create": {
                User u = requireRole(ex,
"manager", "admin");
                if (u != null) create(ex, u);
                break;
            }
        }
    }
}

```

```

    }
    default: notFound(ex, null);
}
}

private void show(HttpExchange ex, User
u) throws Exception {
    StringBuilder b = new
StringBuilder("<h1>Meetings</h1>");

    b.append(HtmlUtil.flash(FormParser.query(e
x).get("msg"),
        "scheduled", "Meeting
scheduled.",
        "!conflict", "That time conflicts with
an existing meeting for you or one of the
invitees.",
        "invitee", "One or more invitees
are not active users. Use their usernames,
separated by commas.",
        "invalid", "Please provide a title
and a valid date and time."));
    if (u.canManage()) {
        b.append("<div
class='card'><h2>Schedule
Meeting</h2><form method='POST'
action='/meetings/create'>")
        .append("<label>Title</label><input
type='text' name='title' required>")
        .append("<label>Date &
Time</label><input type='datetime-local'
name='datetime' required>")
        .append("<label>Invitees
(comma-separated usernames)</label>")
        .append("<input type='text'
name='invitees' placeholder='jdoe,
asmith'>")
        .append("<button
type='submit'>Schedule</button></form></d
iv>");
    }
    b.append("<div
class='card'><h2>Scheduled</h2><table><t

```

```

r><th>When</th><th>Title</th><th>Organis
er</th><th>Invitees</th></tr>");
    for (Meeting m : meetings.listAll()) {
        boolean visible = u.isAdmin() ||
m.createdById == u.id ||
MeetingDao.split(m.invitees).contains(u.use
rname);
        if (!visible) continue;

b.append("<tr><td>").append(HtmlUtil.fmt(m
.meetingTime)).append("</td><td>").append
(HtmlUtil.esc(m.title))
.append("</td><td>").append(HtmlUtil.esc(
m.createdBy)).append("</td><td>")

.append(HtmlUtil.esc(m.invitees)).append("
</td></tr>");
    }
    b.append("</table></div>");
    sendHtml(ex, 200,
HtmlUtil.page("Meetings", u, b.toString()));
}

private void create(HttpExchange ex,
User u) throws Exception {
    if (!isPost(ex)) { redirect(ex,
"/meetings"); return; }
    Map<String, String> f =
FormParser.form(ex);
    String title = f.getDefault("title",
"").trim();
    Timestamp time;
    try {
        time =
Timestamp.valueOf(LocalDateTime.parse(f.
getDefault("datetime", "")));
    } catch (RuntimeException e) {
        redirect(ex,
"/meetings?msg=invalid");
        return;
    }
    if (title.isEmpty()) { redirect(ex,
"/meetings?msg=invalid"); return; }

```

```

Set<String> invitees =
MeetingDao.split(f.getDefault("invitees",
""));
    for (String name : invitees) {
        User inv =
USERS.findByUsername(name);
        if (inv == null ||
!inv.status.equals("active")) { redirect(ex,
"/meetings?msg=invitee"); return; }
    }
    if (meetings.hasConflict(time, u.id,
invitees)) { redirect(ex,
"/meetings?msg=conflict"); return; }

    meetings.insert(title, time, String.join(",
", invitees), u.id);
    redirect(ex,
"/meetings?msg=scheduled");
}
}

```

ViolationHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.ViolationDao;
import com.ocs.model.User;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;

public class ViolationHandler extends
BaseHandler {
    private final ViolationDao violations = new
ViolationDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        if (!path(ex).equals("/violations/report"))
{ notFound(ex, null); return; }
        User u = requireLogin(ex);
        if (u == null) return;

        if (isPost(ex)) {

```

```

        String d =
FormParser.form(ex).getOrDefault("descripti
on", "").trim();
        if (d.isEmpty()) { redirect(ex,
"/violations/report?msg=invalid"); return; }
        violations.insert(u.id, d);
        redirect(ex,
"/violations/report?msg=sent");
        return;
    }
    String body = "<h1>Report an
Issue</h1>"
        +
HtmlUtil.flash(FormParser.query(ex).get("ms
g"),
        "sent", "Your report was submitted
to the administrators.",
        "!invalid", "Please describe the
issue.")
        + "<div class='card'><form
method='POST' action='/violations/report'>"
        +
"<label>Description</label><textarea
name='description' rows='4' required "
        + "placeholder='Describe the
issue or violation'></textarea>"
        + "<button type='submit'>Submit
Report</button></form></div>";
    sendHtml(ex, 200,
HtmlUtil.page("Report Issue", u, body));
    }
}

```

AdminHandler.java:

```

package com.ocs.handler;
import com.ocs.dao.LoginHistoryDao;
import com.ocs.dao.ViolationDao;
import com.ocs.model.LoginRecord;
import com.ocs.model.User;
import com.ocs.model.Violation;
import com.ocs.util.FormParser;
import com.ocs.util.HtmlUtil;
import
com.sun.net.httpserver.HttpExchange;

```

```

import java.util.List;
import java.util.Map;
import java.util.Set;

public class AdminHandler extends
BaseHandler {
    private static final Set<String> ROLES =
Set.of("employee", "manager", "admin");
    private static final Set<String>
STATUSES = Set.of("active", "suspended",
"deactivated");
    private final ViolationDao violations = new
ViolationDao();
    private final LoginHistoryDao logs = new
LoginHistoryDao();

    @Override
    protected void route(HttpExchange ex)
throws Exception {
        User u = requireRole(ex, "admin");
        if (u == null) return;
        switch (path(ex)) {
            case "/admin/users": users(ex, u);
            break;
            case "/admin/users/validate":
            validate(ex, u); break;
            case "/admin/users/status":
            status(ex, u); break;
            case "/admin/users/role": role(ex, u);
            break;
            case "/admin/violations":
            violationList(ex, u); break;
            case "/admin/violations/resolve":
            resolve(ex, u); break;
            case "/admin/history": history(ex, u);
            break;
            default: notFound(ex, u);
        }
    }

    private void users(HttpExchange ex, User
me) throws Exception {

```

```

        StringBuilder b = new
        StringBuilder("<h1>User Validation &
        Access</h1>");

        b.append(HtmlUtil.flash(FormParser.query(e
        x).get("msg"), "done", "Change saved.",
        "!invalid", "That change is not allowed."));
        b.append("<div
        class='card'><h2>Pending
        Registrations</h2>")
        .append("<table><tr><th>Name</th><th>Us
        ername</th><th>Role</th><th></th></tr>");
        List<User> pending =
        USERS.findByStatus("pending");
        for (User x : pending) {
        b.append("<tr><td>").append(HtmlUtil.esc(x
        .name)).append("</td><td>").append(HtmlU
        til.esc(x.username))
        .append("</td><td>").append(HtmlUtil.esc(x
        .role)).append("</td><td>")
        .append(HtmlUtil.postForm("/admin/users/v
        alidate", "Approve", "btn-small", "id", "" +
        x.id, "action", "approve"))
        .append(" ")
        .append(HtmlUtil.postForm("/admin/users/v
        alidate", "Reject", "btn-small btn-danger",
        "id", "" + x.id, "action", "reject"))
        .append("</td></tr>");
        }
        if (pending.isEmpty())
        b.append("<tr><td colspan='4'>No pending
        registrations.</td></tr>");
        b.append("</table></div>");
        b.append("<div class='card'><h2>All
        Users</h2><table><tr><th>Name</th><th>
        Username</th><th>Role</th><th>Status</t
        h><th>Actions</th></tr>");
        for (User x : USERS.findAll()) {

        b.append("<tr><td>").append(HtmlUtil.esc(x
        .name)).append("</td><td>").append(HtmlU
        til.esc(x.username))
        .append("</td><td>").append(HtmlUtil.esc(x
        .role)).append("</td><td><span

```

```

        class='badge badge-")
        .append(HtmlUtil.esc(x.status)).append("<")
        .append(HtmlUtil.esc(x.status)).append("</s
        pan></td><td>");
        if (x.id == me.id) {
        b.append("<span
        class='muted'>(you)</span>");
        } else if (!x.status.equals("pending")
        && !x.status.equals("rejected")) {
        if (x.status.equals("active")) {
        b.append(HtmlUtil.postForm("/admin/users/
        status", "Suspend", "btn-small btn-danger",
        "id", "" + x.id, "status",
        "suspended")).append(" ");
        b.append(HtmlUtil.postForm("/admin/users/
        status", "Deactivate", "btn-small
        btn-danger", "id", "" + x.id, "status",
        "deactivated")).append(" ");
        } else {
        b.append(HtmlUtil.postForm("/admin/users/
        status", "Reactivate", "btn-small", "id", "" +
        x.id, "status", "active")).append(" ");
        }
        b.append("<form method='POST'
        action='/admin/users/role'
        class='inline'><input type='hidden' name='id'
        value=''>
        .append(x.id).append("<select name='role'
        style='display:inline;width:auto;'>");
        for (String r : new
        String[]{"employee", "manager", "admin"}) {
        b.append("<option
        value=''>").append(r).append("<").append(r.eq
        uals(x.role) ? " selected" :
        "").append(">").append(r).append("</option
        >");
        }
        b.append("</select><button
        class='btn-small' type='submit'>Set
        Role</button></form>");
        }
        b.append("</td></tr>");
        }
        b.append("</table></div>");

```

```

        sendHtml(ex, 200,
        HtmlUtil.page("Users", me, b.toString()));
    }

```

```

    private void validate(HttpExchange ex,
    User me) throws Exception {
        if (!isPost(ex)) { redirect(ex,
        "/admin/users"); return; }
        Map<String, String> f =
        FormParser.form(ex);
        int id = FormParser.toInt(f.get("id"), -1);
        boolean approve =
        "approve".equals(f.get("action"));
        User target = USERS.findById(id);
        if (target == null ||
        !target.status.equals("pending")) {
            redirect(ex, "/admin/users?msg=invalid");
            return; }
        USERS.updateStatus(id, approve ?
        "active" : "rejected");
        logs.logAdminAction(me.id, approve ?
        "APPROVE_USER" : "REJECT_USER",
        target.username);
        redirect(ex,
        "/admin/users?msg=done");
    }

```

```

    private void status(HttpExchange ex,
    User me) throws Exception {
        if (!isPost(ex)) { redirect(ex,
        "/admin/users"); return; }
        Map<String, String> f =
        FormParser.form(ex);
        int id = FormParser.toInt(f.get("id"), -1);
        String status = f.getDefault("status",
        "");
        User target = USERS.findById(id);
        if (target == null || id == me.id ||
        !STATUSES.contains(status)) { redirect(ex,
        "/admin/users?msg=invalid"); return; }
        USERS.updateStatus(id, status);
        logs.logAdminAction(me.id,
        "SET_STATUS", target.username + " -> " +
        status);
    }

```

```

        redirect(ex,
        "/admin/users?msg=done");
    }

```

```

    private void role(HttpExchange ex, User
    me) throws Exception {
        if (!isPost(ex)) { redirect(ex,
        "/admin/users"); return; }
        Map<String, String> f =
        FormParser.form(ex);
        int id = FormParser.toInt(f.get("id"), -1);
        String role = f.getDefault("role", "");
        User target = USERS.findById(id);
        if (target == null || id == me.id ||
        !ROLES.contains(role)) { redirect(ex,
        "/admin/users?msg=invalid"); return; }
        USERS.updateRole(id, role);
        logs.logAdminAction(me.id,
        "SET_ROLE", target.username + " -> " +
        role);
        redirect(ex,
        "/admin/users?msg=done");
    }

```

```

    private void violationList(HttpExchange
    ex, User me) throws Exception {
        StringBuilder b = new
        StringBuilder("<h1>Violations</h1><div
        class='card'><table>")
        .append("<tr><th>Reporter</th><th>Descrip
        tion</th><th>Reported</th><th>Status</th>
        <th></th></tr>");
        List<Violation> list = violations.listAll();
        for (Violation v : list) {
            b.append("<tr><td>").append(HtmlUtil.esc(v.
            reporter)).append("</td><td>").append(Html
            Util.esc(v.description))
            .append("</td><td>").append(HtmlUtil.fmt(v.
            reportedAt)).append("</td><td><span
            class='badge badge-")
    }

```

```

.append(HtmlUtil.esc(v.status)).append(">")
.append(HtmlUtil.esc(v.status)).append("</span></td><td>");
    if (v.status.equals("open")) {

b.append(HtmlUtil.postForm("/admin/violations/resolve", "Mark Resolved", "btn-small",
"id", "" + v.id));
    }
    b.append("</td></tr>");
}
if (list.isEmpty()) b.append("<tr><td colspan='5'>No reports.</td></tr>");
b.append("</table></div>");
sendHtml(ex, 200,
HtmlUtil.page("Violations", me,
b.toString()));
}

private void resolve(HttpExchange ex,
User me) throws Exception {
    if (isPost(ex)) {
        int id =
FormParser.toInt(FormParser.form(ex).get("id"), -1);
        violations.resolve(id);
        logs.logAdminAction(me.id,
"RESOLVE_VIOLATION", "violation #" + id);
    }
    redirect(ex, "/admin/violations");
}

private void history(HttpExchange ex,
User me) throws Exception {
    StringBuilder b = new
StringBuilder("<h1>Login & Audit
Logs</h1><div class='card'><h2>All
Logins</h2>")

.append("<table><tr><th>Username</th><th>Time</th><th>IP address</th></tr>");
    for (LoginRecord r : logs.all()) {

```

```

b.append("<tr><td>").append(HtmlUtil.esc(r.
username)).append("</td><td>").append(HtmlUtil.fmt(r.loginTime))

.append("</td><td>").append(HtmlUtil.esc(r.
ipAddress)).append("</td></tr>");
    }
    b.append("</table></div><div
class='card'><h2>Administrator
Actions</h2>")

.append("<table><tr><th>Time</th><th>Admin</th><th>Action</th><th>Details</th></tr>");
    for (String[] row :
logs.recentAdminActions(200)) {

b.append("<tr><td>").append(HtmlUtil.esc(r
ow[0])).append("</td><td>").append(HtmlUtil.esc(row[1]))

.append("</td><td>").append(HtmlUtil.esc(r
ow[2])).append("</td><td>").append(HtmlUtil.esc(row[3])).append("</td></tr>");
    }
    b.append("</table></div>");
    sendHtml(ex, 200,
HtmlUtil.page("Logs", me, b.toString()));
}
}

```

HtmlUtil.java:

```

package com.ocs.util;
import com.ocs.model.User;
import java.sql.Timestamp;
import java.time.format.DateTimeFormatter;

public final class HtmlUtil {
    private static final DateTimeFormatter TS
=
DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");

```

```

private HtmlUtil() { }
public static String esc(String s) {
    if (s == null) return "";
    return s.replace("&",
"&amp;").replace("<", "&lt;").replace(">",
"&gt;").replace(""", "&quot;").replace("'",
"&#39;");
}

```

```

public static String fmt(Timestamp t) {
    return t == null ? "" :
t.toLocalDateTime().format(TS);
}

```

```

public static String notice(String text) {
return "<div class='notice'>" + esc(text) +
"</div>"; }
public static String error(String text) {
return "<div class='error'>" + esc(text) +
"</div>"; }

```

```

public static String flash(String code,
String... spec) {
    if (code == null) return "";
    for (int i = 0; i + 1 < spec.length; i += 2)
{
        boolean err = spec[i].startsWith("!");
        String c = err ? spec[i].substring(1) :
spec[i];
        if (c.equals(code)) return err ?
error(spec[i + 1]) : notice(spec[i + 1]);
    }
    return "";
}

```

```

public static String postForm(String
action, String label, String cssClass,
String... kv) {
    StringBuilder s = new
StringBuilder("<form method='POST'
action='").append(esc(action)).append("'
class='inline'>");
    for (int i = 0; i + 1 < kv.length; i += 2) {

```

```

        s.append("<input type='hidden'
name='").append(esc(kv[i])).append("'
value='").append(esc(kv[i +
1])).append(">");
    }
    return s.append("<button type='submit'
class='").append(esc(cssClass)).append(">")
");
}

```

```

.append(esc(label)).append("</button></for
m>").toString();
}

```

```

private static void link(StringBuilder n,
String href, String text) {
    n.append("<a
href='").append(href).append(">").append(t
ext).append("</a>");
}

```

```

public static String page(String title, User
u, String body) {
    StringBuilder n = new
StringBuilder("<div class='nav'><div
class='nav-brand'>OrgComm</div><div
class='nav-links'>");
    if (u != null) {
        link(n, "/dashboard", "Dashboard");
        link(n, "/messages", "Messages");
        link(n, "/calendar", "Calendar");
        link(n, "/announcements",
"Announcements");
        link(n, "/polls", "Polls");
        link(n, "/meetings", "Meetings");
        link(n, "/history", "My Logins");
        if (u.isAdmin()) {
            link(n, "/admin/users", "Users");
            link(n, "/admin/violations",
"Violations");
            link(n, "/admin/history", "Logs");
        } else {
            link(n, "/violations/report", "Report
Issue");
        }
    }
}

```

```

        n.append("<span
class='nav-user'>").append(esc(u.name)).ap
pend("
").append(esc(u.role)).append("</span>");
        n.append(postForm("/logout",
"Logout", "link-btn"));
    }
    n.append("</div></div>");
    return "<!DOCTYPE
html><html><head><meta charset='utf-8'>"
        + "<meta name='viewport'
content='width=device-width,
initial-scale=1'>"
        + "<title>" + esc(title) +
"</title><link rel='stylesheet'
href='/style.css'></head><body>"
        + n + "<div class='container'>" +
body + "</div></body></html>";
    }
}

```

FormParser.java:

```

package com.ocs.util;
import
com.sun.net.httpserver.HttpExchange;
import java.io.IOException;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.LinkedHashMap;
import java.util.Map;

public final class FormParser {
    private FormParser() { }
    public static Map<String, String>
form(HttpExchange ex) throws IOException
{    return parse(new
String(ex.getRequestBody().readAllBytes(),
StandardCharsets.UTF_8));
    }

    public static Map<String, String>
query(HttpExchange ex) {
        return
parse(ex.getRequestURI().getRawQuery());
    }
}

```

```

    }

    public static Map<String, String>
parse(String s) {
        Map<String, String> map = new
LinkedHashMap<>();
        if (s == null || s.isEmpty()) return map;
        for (String pair : s.split("&")) {
            int i = pair.indexOf('=');
            if (i < 0) continue;
            try {

map.put(URLDecoder.decode(pair.substring
(0, i), StandardCharsets.UTF_8),

URLDecoder.decode(pair.substring(i + 1),
StandardCharsets.UTF_8));
            } catch (IllegalArgumentException
ignored) { }
        }
        return map;
    }

    public static int toInt(String s, int def) {
        try { return Integer.parseInt(s.trim()); }
        catch (RuntimeException e) { return def; }
    }
}

```

PasswordHasher.java:

```

package com.ocs.util;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import
java.security.GeneralSecurityException;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;

public final class PasswordHasher {
    private static final int ITERATIONS =
120_000;
    private static final int KEY_BITS = 256;
}

```

```
private static final SecureRandom RNG =
new SecureRandom();
```

```
private PasswordHasher() { }
public static String hash(String password)
{
    byte[] salt = new byte[16];
    RNG.nextBytes(salt);
    byte[] h =
derive(password.toCharArray(), salt,
ITERATIONS);
    Base64.Encoder enc =
Base64.getEncoder();
    return "pbkdf2" + ITERATIONS + "" +
enc.encodeToString(salt) + "$" +
enc.encodeToString(h);
}
```

```
public static boolean verify(String
password, String stored) {
    try {
        String[] p = stored.split("\\$");
        if (p.length != 4 ||
!p[0].equals("pbkdf2")) return false;
        int iter = Integer.parseInt(p[1]);
        byte[] salt =
Base64.getDecoder().decode(p[2]);
        byte[] expected =
Base64.getDecoder().decode(p[3]);
        byte[] actual =
derive(password.toCharArray(), salt, iter);
        return
MessageDigest.isEqual(expected, actual);
    } catch (RuntimeException e) {
        return false;
    }
}
```

```
private static byte[] derive(char[] pw,
byte[] salt, int iter) {
    try {
        SecretKeyFactory f =
SecretKeyFactory.getInstance("PBKDF2With
hHmacSHA256");
```

```
        return f.generateSecret(new
PBEKeySpec(pw, salt, iter,
KEY_BITS)).getEncoded();
    } catch (GeneralSecurityException e) {
        throw new IllegalStateException(e);
    }
}
```

SessionManager.java:

```
package com.ocs.util;
import com.ocs.Database;
import java.security.SecureRandom;
import java.util.Base64;
import java.util.Map;
import
java.util.concurrent.ConcurrentHashMap;
```

```
public final class SessionManager {
    private static final class Session {
        final int userId;
        volatile long lastAccess =
System.currentTimeMillis();
        Session(int userId) { this.userId =
userId; }
    }
```

```
    private static final Map<String, Session>
SESSIONS = new
ConcurrentHashMap<>();
    private static final SecureRandom RNG =
new SecureRandom();
```

```
private SessionManager() { }
```

```
public static String create(int userId) {
    byte[] b = new byte[32];
    RNG.nextBytes(b);
    String token =
Base64.getUrlEncoder().withoutPadding().e
ncodeToString(b);
    SESSIONS.put(token, new
Session(userId));
    return token;
```

```

    }

    public static Integer getUserId(String
token) {
        if (token == null) return null;
        Session s = SESSIONS.get(token);
        if (s == null) return null;
        long timeout =
Database.getInt("session.timeout.minutes",
30) * 60_000L;
        long now = System.currentTimeMillis();
        if (now - s.lastAccess > timeout) {
            SESSIONS.remove(token);
            return null;
        }
        s.lastAccess = now;
        return s.userId;
    }

    public static void destroy(String token) {
        if (token != null)
SESSIONS.remove(token);
    }
}

```

pom.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<project
xmlns="http://maven.apache.org/POM/4.0.0
"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.o
rg/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.x
sd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.ocs</groupId>
    <artifactId>organizational-communication-sy
stem</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>
    <name>Organizational Communication
System</name>

```

```

    <properties>
    <maven.compiler.source>17</maven.compil
er.source>
    <maven.compiler.target>17</maven.compil
er.target>
    <project.build.sourceEncoding>UTF-8</proj
ect.build.sourceEncoding>
    </properties>
    <dependencies>
    <dependency>
        <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
        <version>8.3.0</version>
    </dependency>
    </dependencies>
    <build>
    <finalName>ocs</finalName>
    <plugins>
    <plugin>

    <groupId>org.apache.maven.plugins</grou
pId>
    <artifactId>maven-shade-plugin</artifactId>
        <version>3.5.1</version>
        <executions>
        <execution>
            <phase>package</phase>
            <goals><goal>shade</goal></goals>
            <configuration>
    <createDependencyReducedPom>false</cr
eateDependencyReducedPom>
            <transformers>
            <transformer
implementation="org.apache.maven.plugins
.shade.resource.ManifestResourceTransfor
mer">
    <mainClass>com.ocs.Main</mainClass>
            </transformer>
            <transformer
implementation="org.apache.maven.plugins
.shade.resource.ServicesResourceTransfor
mer"/>
            </transformers>
        </configuration>
    </plugin>
    </plugins>
    </build>
    </project>

```

```

        </execution>
      </executions>
    </plugin>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.1.1</version>
      <configuration>
        <mainClass>com.ocs.Main</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

style.css:

```

*{box-sizing:border-box;}
body{font-family:Arial,Helvetica,sans-serif;margin:0;background:#f4f6f8;color:#222;}
.nav{background:#20345a;color:#fff;padding:10px 20px;display:flex;justify-content:space-between;align-items:center;flex-wrap:wrap;}
.nav-brand{font-weight:bold;font-size:18px;}
.nav-links{display:flex;align-items:center;flex-wrap:wrap;}
.nav-links a,.link-btn{color:#dce6f5;text-decoration:none;margin-right:14px;font-size:14px;background:none;border:none;padding:0;cursor:pointer;}
.nav-links a:hover,.link-btn:hover{color:#fff;text-decoration:underline;background:none;}
.nav-user{color:#9fb3d6;margin-right:14px;font-size:13px;}
.container{max-width:900px;margin:20px auto;padding:0 16px;}
.card{background:#fff;border:1px solid #dde3ea;border-radius:6px;padding:16px 20px;margin-bottom:16px;}
h1{font-size:22px;}
h2{font-size:18px;margin-top:0;}

```

```

table{width:100%;border-collapse:collapse;margin-top:8px;}
th,td{text-align:left;padding:8px;border-bottom:1px solid #eee;font-size:14px;}
input[type=text],input[type=password],input[type=date],input[type=datetime-local],textarea,select{width:100%;padding:8px;margin:6px 0 12px;border:1px solid #ccc;border-radius:4px;font-size:14px;}
label{font-size:13px;font-weight:bold;color:#444;}
button,.btn{background:#20345a;color:#fff;border:none;padding:8px 16px;border-radius:4px;cursor:pointer;font-size:14px;text-decoration:none;display:inline-block;}
button:hover,.btn:hover{background:#16264a;}
.btn-danger{background:#a33;}
.btn-danger:hover{background:#822;}
.btn-small{padding:4px 10px;font-size:12px;}
form.inline{display:inline;}
.badge{padding:2px 8px;border-radius:10px;font-size:12px;color:#fff;}
.badge-pending,.badge-open{background:#c08b1e;}
.badge-active,.badge-resolved{background:#2e8b57;}
.badge-suspended,.badge-locked{background:#a33;}
.badge-rejected,.badge-deactivated{background:#777;}
.msg{padding:8px;margin:4px 0;border-left:3px solid #20345a;background:#f8f9fb;font-size:13px;}
.tag{font-size:11px;background:#e3e9f4;border-radius:8px;padding:1px 7px;margin-left:6px;}
.muted{color:#777;font-size:13px;}
.row{display:flex;gap:16px;flex-wrap:wrap;}
.row>.card{flex:1;min-width:280px;}

```

SCREENSHOTS:

OrgComm

Registration submitted. An administrator must approve your account before you can log in.

Login
Username
admin
Password
.....
Login

Register
Full name
Username
Department
Password
Role
Employee
Register

Messages

Compose
To
Administrator (admin)
Type
General message
Message
☐ Send anonymously
Anonymity cannot be changed after the message is sent.
Send

Inbox
No messages received.

Sent
No messages sent.

My Login History

Time	IP address
2026-10-01 09:47:06	0:0:0:0:0:0:1
2026-10-01 09:45:03	0:0:0:0:0:0:1
2026-10-01 09:42:15	0:0:0:0:0:0:1

Calendar

Add Event

Title

Date

dd/mm/yyyy

Description

Add

Upcoming Events

Date	Title	Description	
2026-10-08	Quarterly Review Meeting	Second conference hall, 10:00 AM	Delete
2026-10-22	Company Sports Day	Main ground	Delete

omm

Dashboard Messages Calendar Announcements Polls Meetings My Logins Report Issue Zosangi Renthlei (employee)

Calendar

Upcoming Events

Date	Title	Description
2026-10-08	Quarterly Review Meeting	Second conference hall, 10:00 AM
2026-10-22	Company Sports Day	Main ground

Announcements

New Announcement

Title

Body

Post

Feed

Welcome to OrgComm

Use this portal for feedback, requests, announcements, polls and meeting schedules.

System - 2026-10-01 09:47:30

Announcements

Feed

Welcome to OrgComm

Use this portal for feedback, requests, announcements, polls and meeting schedules.

System - 2026-10-01 09:47:30

Polls

New Poll

Question

Options (comma-separated)

Create

Preferred time for the monthly all-hands meeting?

You already voted.

Option	Votes
Morning	0
Afternoon	1

Total votes: 1

Polls

Preferred time for the monthly all-hands meeting?

☐ Morning
☐ Afternoon

Vote

Option	Votes
Morning	0
Afternoon	1

Total votes: 1

Meetings

Schedule Meeting

Title

Date & Time

Invitees (comma-separated usernames)

Schedule

Scheduled

When	Title	Organiser	Invitees
2026-10-05 15:30:00	Sports Event	stephen065	zosangi065

Meetings

Scheduled

When	Title	Organiser	Invitees
2026-10-05 15:30:00	Sports Event	stephen065	zosangi065

OrgComm [Dashboard](#) [Messages](#) [Calendar](#) [Announcements](#) [Polls](#) [Meetings](#) [My Logins](#) [Report Issue](#) [Zosangi Renthlei \(employee\)](#) [Logout](#)

Report an Issue

Description

Message encryption

Submit Report

OrgComm [Dashboard](#) [Messages](#) [Calendar](#) [Announcements](#) [Polls](#) [Meetings](#) [My Logins](#) [Users](#) [Violations](#) [Logs](#) [Administrator \(admin\)](#) [Logout](#)

Violations

Reporter	Description	Reported	Status
zosangi065	Message encryption	2026-10-01 09:54:57	resolved

Login & Audit Logs

All Logins

Username	Time	IP address
admin	2026-10-01 09:55:39	0:0:0:0:0:0:1
stephen065	2026-10-01 09:55:16	0:0:0:0:0:0:1
zosangi065	2026-10-01 09:52:25	0:0:0:0:0:0:1
admin	2026-10-01 09:51:42	0:0:0:0:0:0:1
admin	2026-10-01 09:47:06	0:0:0:0:0:0:1
admin	2026-10-01 09:45:03	0:0:0:0:0:0:1
admin	2026-10-01 09:42:15	0:0:0:0:0:0:1

Administrator Actions

Time	Admin	Action	Details
2026-10-01 09:55:50.0	admin	RESOLVE_VIOLATION	violation #1
2026-10-01 09:52:08.0	admin	APPROVE_USER	stephen065
2026-10-01 09:50:50.0	admin	APPROVE_USER	zosangi065

User Validation & Access

Pending Registrations

Name	Username	Role	
Stephen Lalruatpuia	stephen065	manager	Approve Reject

All Users

Name	Username	Role	Status	Actions
Administrator	admin	admin	active	(you)
Zosangi Renthlei	zosangi065	employee	active	Suspend Deactivate employee Set Role
Stephen Lalruatpuia	stephen065	manager	pending	

Welcome, Administrator

Messages
Send and view messages, complaints, or feedback.
[Open](#)

Calendar
View company and department events.
[Open](#)

Announcements
[Open](#)

Polls
[Open](#)

Meetings
[Open](#)

My Login History
See your past login activity.
[Open](#)

User Validation & Access
[Open](#)

Violations
[Open](#)

Login & Audit Logs
[Open](#)

Welcome, Zosangi Renthlei

Messages
Send and view messages, complaints, or feedback.
[Open](#)

Calendar
View company and department events.
[Open](#)

Announcements
[Open](#)

Polls
[Open](#)

Meetings
[Open](#)

My Login History
See your past login activity.
[Open](#)

Report an Issue
[Open](#)

Appendix D: To be Determined List

This section lists requirements and decisions that remain unresolved as of this version of the document. Each item should be finalized and the corresponding section updated before the system is implemented in full or submitted for final review.

Sl. No.	Item	Related Section	Description
1	Server sizing/specifications	3.2 Hardware Interfaces	Specific CPU, RAM, and storage requirements for the application and database servers have not been determined; dependent on deployment scale.
2	Concurrent user capacity	5.1 Performance Requirements	The exact number of concurrent users the system must support in production has not been confirmed with stakeholders (currently estimated at 100).
3	Authentication provider	3.3 Software Interfaces	Whether the system will use an internal authentication module or a third-party identity provider (e.g., OAuth2, SSO) is undecided.
4	Real-time update mechanism	3.4 Communications Interfaces	Whether notifications will be delivered via WebSocket (true real-time) or a polling mechanism has not been finalized.
5	SMS gateway provider	3.4 Communications Interfaces	Whether SMS notifications will be implemented at all, and if so, which SMS gateway provider will be used, is undecided.
6	Department-level scoping	2. Overall Description / Business Rules	Whether calendars, policies, and messaging will be scoped per department or shared company-wide has not been finalized (see BR-8).
7	Password recovery flow	3.1 User Interfaces	The exact mechanism for password reset (email link, security questions, admin-assisted reset) has not been designed.
8	Message type enforcement	4.1/4.2	Whether message categories (feedback, complaint, request) will be enforced as distinct data types or remain a conceptual/UI-only classification is undecided.
9	Data retention period	6.2 Data Retention	The specific retention period (e.g., 6 months, 1 year) for messages, login history, and violation records has not been set by organizational policy.

10	Legal disclosure process for anonymous senders	6.1 Legal and Regulatory	The formal process (who authorizes it, what documentation is required) for disclosing an anonymous sender's identity under legal compulsion has not been defined.
11	Multi-language support	3.1.5 General UI Requirements	Whether multi-language support will be implemented in this version or deferred to a future release has not been decided.
12	Failed login lockout threshold	5.3 Security Requirements	The specific number of failed login attempts before an account is locked/flagged (SC-6) has not been set.
13	Session timeout duration	5.3 Security Requirements	The exact period of inactivity before a session automatically expires (SC-5) has not been determined.
14	Violation escalation path	4.1 Administrator Use Case	Whether violations can be escalated beyond the Administrator (e.g., to HR or executive leadership) is not yet defined.